

Associativity of Multiplication Is Hard for Resolution

Vincent Liew
Category Labs
vliew@vliew.com

October 6, 2026

Abstract

SAT solvers are empirically known to perform poorly when reasoning about multiplication. Yet for over a decade we have lacked a theoretical explanation for this phenomenon. CDCL SAT solvers implicitly search for resolution proofs, and no lower bound on proof size has ruled out the existence of short proofs that solvers simply fail to find.

We give the first lower bound of this kind by showing that general resolution proofs of the associativity of n -bit multiplication require size $2^{\Omega((n/\log n)^{1/4})}$. This lower bound holds for a broad class of multiplier encodings based on partial product summation, including the standard array and Wallace-tree multipliers used to bit-blast multiplication in SMT solvers. This result resolves an open problem of Beame and Liew.

The proof constructs a reduction from a perfect-matching principle on bounded-degree bipartite expander graphs to multiplier associativity. Itsykson, Slabodkin, and Sokolov proved that this principle is hard for resolution. The lower bound for multiplier associativity follows.

The same reduction, when combined with Håstad's recent lower bound for the perfect-matching principle of the odd grid, yields an exponential lower bound for multiplier associativity in the stronger bounded-depth Frege proof systems.

1 Introduction

Why do SAT solvers struggle with multiplication? Despite dramatic leaps in efficiency in recent decades, the inability of SAT solvers to reason about integer multiplication has remained a major obstacle for many applications of SAT and SMT solving, among them bounded model checking [BBK⁺26], equivalence checking of arithmetic hardware [KBK19, BKKS25], and the verification of cryptographic code and smart contracts [FLS⁺19, ZIM⁺22, REK⁺25]. The persistence of this difficulty is illustrated by a recent evaluation on multiplier equivalence-checking benchmarks submitted to the 2025 SAT Competition [BKKS25], where the modern SAT solver KISSAT [BFFH20] did not outperform the twenty-year-old solver MINISAT [ES04].

This weakness is felt most acutely in bit-vector solving, where SAT solvers serve as the core reasoning engine. Bit-vector solvers ultimately handle arithmetic by *bit-blasting*: each bit becomes a Boolean variable and each operation is represented by a Boolean circuit, so a bit-vector formula involving multiplication reduces to a SAT instance containing a multiplier circuit. While bit-blasting is typically effective for bit-vector formulas containing, for example, addition, comparisons, and bitwise operations, performance collapses on formulas requiring substantial reasoning about multiplication [Lie20, BBK⁺26].

Practitioners have responded by engineering around this weakness, turning to forms of reasoning better suited to multiplication: multiplier verification via Gröbner bases [SGK⁺16, RBK17, KBK19, MGD19, KBBN22]; word-level algebraic decision procedures such as PolySAT [REK⁺25], int-blasting [ZIM⁺22], and cvc5’s finite-field solver [OKTB23]; as well as rewriting and abstraction layers designed to keep multiplication away from the underlying SAT solver [NPZ24]. The reliance on rewriting runs especially deep [NPR⁺21]: before bit-blasting, the rewrite engines of all major bit-vector solvers (e.g., Bitwuzla [NP23], cvc5 [BBB⁺22], and Z3 [dMB08]) normalize arithmetic terms modulo commutativity and associativity of multiplication, so that the underlying SAT solver never has to reason about these properties.

Proof complexity provides the standard framework for explaining this kind of empirical hardness. In order for a CDCL SAT solver to determine that a formula is unsatisfiable, it must implicitly construct a *resolution refutation* of that formula [BKS04]. The running time of a CDCL solver is therefore bounded below by the size of the smallest resolution refutation. A classic example is the pigeonhole principle: Haken’s exponential lower bound [Hak85] explains why state-of-the-art solvers fail to show that $n + 1$ pigeons do not fit into n holes, for modest values of n . In this spirit, Biere conjectured in 2016 that even basic algebraic properties of multiplication circuits, such as commutativity, require exponentially large resolution refutations [Bie16].

Beame and Liew gave a surprising negative answer to this conjecture for the case of degree two ring identities, which include commutativity and distributivity, but not associativity [BL19]. But although they constructed polynomial size proofs for these identities, the concrete size of these proofs was still large. Their best upper bounds, for commutativity (i.e., $\mathbf{xy} = \mathbf{yx}$) and distributivity (i.e., $\mathbf{x(y + z)} = \mathbf{xy + xz}$), scaled as $O(n^6 \log n)$, where n is the bit-width. Follow-up experiments indicated that CDCL SAT solvers do not find these proofs on standard multiplier encodings, even with substantial guidance [Lie20, BBK⁺26]. A different set of experiments in Liew’s thesis suggested that even if SAT solvers did find these proofs, their size would still be prohibitive in practice [Lie20]. So although Beame and Liew’s result resolved Biere’s specific conjecture, it gave limited insight into the broader aim of explaining *why* SAT solvers consistently struggle with multiplication.

Meanwhile, the practical workarounds described above are incomplete, and when they fail, every major solver still falls back to bit-blasting. Moreover, this retreat from CDCL has rested entirely on empirical grounds. Without proven lower bounds, the possibility has remained that these workarounds are unnecessary. Perhaps resolution proofs of practical size exist for these problems,

and current CDCL solvers simply lack the heuristics to find them.

For associativity of multiplication, we show that such small proofs do not exist. In particular, we prove an exponential general resolution lower bound for multiplier associativity, resolving a problem left open by Beame and Liew [BL19]. This is the first resolution lower bound for a natural formula involving multiplication that holds without restricting the order in which variables are resolved. Prior to this result, no such lower bound was known even for tree-like or regular resolution.

Theorem 8.8 (informal). *Let Assoc_n be the CNF formula asserting $(\mathbf{x}\mathbf{y})\mathbf{z} \neq \mathbf{x}(\mathbf{y}\mathbf{z})$ for n -bit input bit-vectors $\mathbf{x}, \mathbf{y}, \mathbf{z}$, with multiplication encoded by an array or Wallace-tree multiplier. Then resolution refutations of Assoc_n require size $2^{\Omega((n/\log n)^{1/4})}$.*

This lower bound is not specific to these two multiplier architectures. It applies to a general class of *strip-local* multiplier encodings, defined in Section 4, which captures most of the standard multipliers used for bit-blasting.

We prove the lower bound by constructing a reduction from the perfect-matching principle on bounded-degree bipartite graphs. The reduction reveals a perfect matching instance embedded inside the associativity formula: we fix $\mathbf{x}$ and $\mathbf{z}$ to sparse bit-vectors whose nonzero positions represent the vertices of the graph, and leave selected bits of $\mathbf{y}$ free to serve as edge variables. We use the bipartite expanders of Itsykson, Slabodkin, and Sokolov [ISS15] (see also [IOSS16]), who proved that this principle on these graphs is hard for resolution. In order to absorb the auxiliary circuit variables of the multiplier encodings, we extend the perfect-matching principle to a *star-local* variant and show that it remains hard for resolution.

Our second result concerns the stronger bounded-depth Frege proof systems, where the lines of a proof are bounded-depth formulas rather than clauses. Håstad recently proved that the perfect-matching principle of the odd grid is hard for these systems [Hå25]. By applying the same reduction as before, with the odd grid in place of the expander, we get the following lower bound.

Theorem 9.5 (informal). *There is a constant $\gamma > 0$ such that for all $d \leq \gamma \log n / \log \log n$, depth d Frege refutations of Assoc_n require size $\exp(n^{\gamma/d})$.*

This result implies an exponential lower bound for resolution as well, but with a smaller exponent than in Theorem 8.8. Moreover, Håstad’s lower bound for the odd grid is significantly more technical than the lower bound for bipartite expanders. For these reasons we work with expanders for the resolution lower bound.

1.1 Related work

The proof complexity of multipliers has been previously studied in the resolution, cutting planes, and polynomial calculus proof systems. However, multiplier associativity in particular has resisted attempts at upper and lower bounds in each of these proof systems.

Resolution. As discussed above, Beame and Liew [BL19] constructed polynomial size resolution proofs for degree two identities, including commutativity and distributivity. On the lower-bound side, [SLVG26] gave the first proof complexity lower bound involving multiplier circuits, but only in the restrictive *partially-ordered resolution* proof system, and only for specific partial orders.

Cutting planes. [LBD⁺20] gave $O(n^2)$ -length cutting planes proofs for a large class of degree two identities. These proofs are optimal, as the multiplier encodings themselves consist of $\Omega(n^2)$ inequalities. They noted, however, that their method does not extend to associativity, whose cutting planes proof complexity remains open.

Polynomial calculus. Gröbner-basis methods, whose derivations are polynomial calculus proofs, were introduced for gate-level multiplier verification [SGK⁺16, RBK17]. These methods were later scaled to large, highly optimized multipliers by Mahzoon, Große, and Drechsler [MGD19], and, with the help of an auxiliary SAT solver, by Kaufmann, Biere, and Kauers [KBK19]. The SAT solver was subsequently eliminated by encoding negated literals with dual variables [KBBN22], so that the resulting procedure yields a single polynomial calculus certificate.

Beyond equivalence checking, Kaufmann, Biere, and Kauers showed that Gröbner-basis methods efficiently verify word-level commutativity of multipliers in practice [KBK20], and that the underlying derivation amounts to an $O(n^2)$ -length polynomial calculus proof of word-level commutativity for array multipliers. Their idea extends to show that word-level ring identities in general, including associativity, admit $O(n^2)$ -length polynomial calculus proofs. However, we do not know whether the corresponding bit-level identities have small proofs. [LBD⁺20] also proved that polynomial calculus requires exponentially many monomials to extract the bit-level consequences of a word-level equality. So for associativity, the word-level equality $(\mathbf{x}\mathbf{y})\mathbf{z} = \mathbf{x}(\mathbf{y}\mathbf{z})$ does not immediately yield a small polynomial calculus proof that the middle output bits agree.

This gap between word-level and bit-level reasoning is also visible in practice: PolySAT [REK⁺25], int-blasting [ZIM⁺22], and cvc5’s finite-field solver [OKTB23] all reason with polynomial constraints at the word level, and each adds dedicated mechanisms for recovering bit-level facts from that reasoning.

Organization. Section 2 collects preliminaries. Section 3 defines the associativity formula Assoc_n . Section 4 defines strip-local multiplier encodings, and shows that array and Wallace-tree multipliers are strip-local. Section 5 introduces the perfect-matching principle $\text{PM}(H)$ and its star-local extension $\text{SPM}(H)$, and proves that the latter is hard for resolution. Section 6 outlines the main steps and ideas of the reduction. Section 7 constructs the key ingredient of the reduction: the restriction ρ . This restriction reveals a perfect matching inside Assoc_n . Section 8 proves the associativity lower bound for resolution (Theorem 8.8) by carrying out the reduction from the star-local extension of perfect matching to the associativity formula. Section 9 carries out the same reduction in bounded-depth Frege systems, with the odd grid in place of the expander, resulting in Theorem 9.5. Section 10 concludes with open problems.

2 Preliminaries

Notation. For integers $a \leq b$, we write $[a, b) = \{a, a + 1, \dots, b - 1\}$. A length n bit-vector is written as $\mathbf{x} = (x_0, \dots, x_{n-1}) \in \{0, 1\}^n$, and represents the integer $\sum_{i < n} x_i 2^i$. Following [BL19], we refer to each multiplier by its output bit-vector, so the multiplier with inputs $\mathbf{x}, \mathbf{y}$ is labeled $\mathbf{x}\mathbf{y}$.

CNF formulas. A *literal* is a Boolean variable or its negation; a *clause* is a disjunction of literals, which we treat as a set. A clause containing complementary literals is a *tautology*. A CNF formula is a conjunction of clauses. The *width* of a clause is the number of literals it contains, and the width $w(\Phi)$ of a CNF Φ is the maximum width of its clauses.

Resolution. The *resolution rule* produces the clause $A \vee B$, called the *resolvent*, from clauses $A \vee x$ and $B \vee \neg x$. A *resolution derivation* is a sequence of clauses, each of which is either an *initial clause* of the input CNF Φ , or follows from two prior clauses via the resolution rule. The *size* of a derivation is the length of the sequence of clauses. A derivation is a *refutation* of Φ if it ends with the empty clause $\perp$. The minimum size of a resolution refutation of Φ is denoted by $\text{size}_{\text{Res}}(\Phi)$.

The *width* of a derivation is the maximum width of its clauses, and $w(\Phi \vdash \perp)$ denotes the minimum width of a resolution refutation of Φ . Resolution is sound and complete: it can only derive clauses that are *implied* by Φ , and conversely, if Φ implies a nontautological clause C , then some subclause of C has a resolution derivation from Φ .

We record the size–width tradeoff of Ben-Sasson and Wigderson in the form we use.

Theorem 2.1 ([BW01]). *Let Φ be an unsatisfiable CNF with N variables. Then*

$$\text{size}_{\text{Res}}(\Phi) \geq \exp\left(\Omega\left(\frac{(w(\Phi \vdash \perp) - w(\Phi))^2}{N}\right)\right).$$

In particular, if Φ has constant width and $O(m)$ variables, and every resolution refutation of Φ has width $\Omega(m)$, then $\text{size}_{\text{Res}}(\Phi) \geq 2^{\Omega(m)}$.

Substitutions. A *substitution* σ maps each variable of a CNF Φ to a constant or to a literal, possibly over a different variable set. We write C^σ or Φ^σ for the result of applying σ to a clause C , or a CNF Φ , respectively. As usual, Φ^σ omits the clauses that have become trivially true.

Restrictions. A *restriction* is the special case of a substitution where every variable maps to a constant or to itself. We denote the result of applying a restriction ρ to a clause C by $C \upharpoonright_\rho$, and the result of applying this restriction to a formula Φ as $\Phi \upharpoonright_\rho$.

Lemma 2.2 (Substitution lemma). *Let σ be a substitution. If a CNF Φ has a resolution refutation of size s , then Φ^σ has a resolution refutation of size at most s .*

Definition 2.3 (Exact multiplier encoding). An *exact multiplier encoding* $\text{Mul}_n(\mathbf{x}, \mathbf{y}; \mathbf{xy})$ is a CNF whose variables are the bits of the n -bit inputs $\mathbf{x}$ and $\mathbf{y}$ together with auxiliary variables called *wires*. Each wire computes a Boolean function of the input bits, in the sense that the satisfying assignments of the CNF are exactly the assignments in which every wire takes the value of its function. The wires include a designated $2n$ -bit *output* $\mathbf{xy}$, computing the product of the input bit-vectors.

Definition 2.4 (Partial products, columns). The *partial products* of two length- n bit-vectors $\mathbf{x}$ and $\mathbf{y}$ are

$$t_{i,j} = x_i \wedge y_j, \quad 0 \leq i, j < n.$$

The product $\mathbf{xy}$ is the weighted sum of these partial products: $\mathbf{xy} = \sum_{i,j} t_{i,j} 2^{i+j}$. The partial product $t_{i,j}$ has *arithmetic weight* 2^{i+j} , and we say that it lies in *column* $i+j$.

Array multiplier. A full adder with inputs a_0, a_1, a_2 each of weight 2^k has outputs

$$s = a_0 \oplus a_1 \oplus a_2, \quad c = \text{MAJ}(a_0, a_1, a_2),$$

of weights 2^k and 2^{k+1} , so the inputs and outputs of a full adder have the same total weight. The n -bit array multiplier, as shown in Figure 1, computes each partial product $t_{i,j}$ by an AND gate and uses a grid of full adders to compute the sum of all the partial products. Observe that this is an exact multiplier encoding.

Bipartite graphs and matchings. Let $G = (U, V, E)$ be a bipartite graph with parts U and V and edges E . For an edge $e \in E$ we write $e = (u_e, v_e)$ with $u_e \in U$ and $v_e \in V$. For a vertex w , we write $E(w)$ for the set of edges incident to w , called the *star* of w ; the degree of w is $|E(w)|$. A *matching* is a set of pairwise disjoint edges; it *covers* a vertex set if every vertex of the set is an endpoint of an edge in the matching.

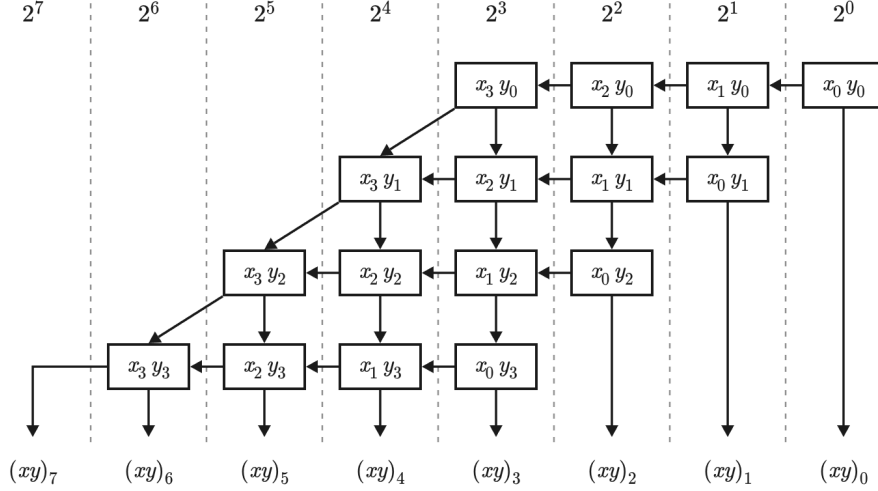


Figure 1: The 4-bit array multiplier. Each box represents a full adder, and adds the partial product $t_{i,j} = x_i \wedge y_j$ to the sum bit arriving from the row above (vertical arrows) and the carry arriving from the previous column (horizontal and diagonal arrows).

3 The associativity formula

The associativity formula Assoc_n contains a circuit of two multipliers outputting $(\mathbf{xy})\mathbf{z}$, a separate circuit of two multipliers outputting $\mathbf{x}(\mathbf{yz})$, and a *miter circuit* [Bra93] which asserts that these two outputs disagree.¹

Definition 3.1 (Associativity formula). Let Mul_n be an exact multiplier encoding. The unsatisfiable formula Assoc_n , shown in Figure 2, consists of the four multiplier encodings

$$\text{Mul}_n(\mathbf{x}, \mathbf{y}; \mathbf{xy}), \quad \text{Mul}_n(\mathbf{y}, \mathbf{z}; \mathbf{yz}),$$

$$\text{Mul}_{2n}(\mathbf{xy}, \mathbf{z}; (\mathbf{xy})\mathbf{z}), \quad \text{Mul}_{2n}(\mathbf{x}, \mathbf{yz}; \mathbf{x}(\mathbf{yz})),$$

along with *miter variables* e_i and *miter clauses* encoding the constraints

$$e_i \leftrightarrow (((\mathbf{xy})\mathbf{z})_i \oplus (\mathbf{x}(\mathbf{yz}))_i), \quad 0 \leq i < 4n,$$

$$e_0 \vee \dots \vee e_{4n-1}.$$

Here the outputs of the inner multipliers, $\mathbf{xy}$ and $\mathbf{yz}$, have $2n$ bits, the n -bit inputs $\mathbf{x}$ and $\mathbf{z}$ to the outer multipliers are zero-padded to $2n$ bits, and the outputs of the outer multipliers $(\mathbf{xy})\mathbf{z}$ and $\mathbf{x}(\mathbf{yz})$ have $4n$ bits.

We refer to $\mathbf{xy}$ and $\mathbf{yz}$ as the *inner* multipliers and $(\mathbf{xy})\mathbf{z}$ and $\mathbf{x}(\mathbf{yz})$ as the *outer* multipliers.

In fixed-width bit-vector arithmetic, multiplier outputs are truncated to the input bit-width. We work with untruncated multipliers for convenience, as the lower bound will apply unchanged to truncated multipliers (see Remark 8.9).

¹The term presumably comes from the shape of the construction: two circuits converging on an XOR gate resemble a bishop's miter, as illustrated by Biere [Bie16, slide 36].

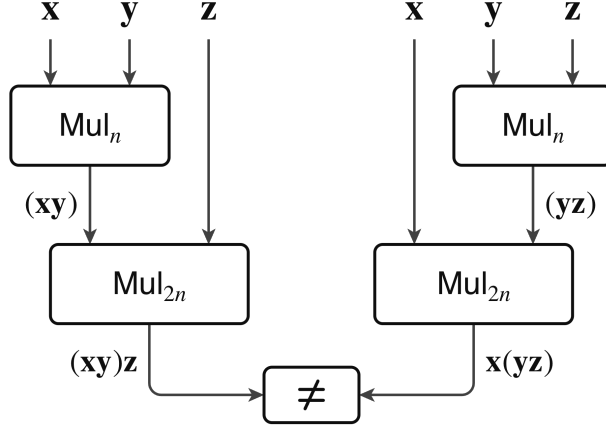


Figure 2: The associativity instance Assoc_n . The two inner multipliers Mul_n compute $\mathbf{xy}$ and $\mathbf{yz}$, the two outer multipliers Mul_{2n} compute $(\mathbf{xy})\mathbf{z}$ and $\mathbf{x}(\mathbf{yz})$, and the miter variables e_i , with the long miter clause $e_0 \vee \dots \vee e_{4n-1}$, assert that $(\mathbf{xy})\mathbf{z} \neq \mathbf{x}(\mathbf{yz})$.

4 Strip-local multipliers

Our Assoc_n lower bound will apply so long as the multiplier encoding used is *strip-local*, a property we will define shortly. Most multiplier encodings based on partial product summation satisfy this property, including standard array and Wallace-tree multipliers.

Definition 4.1 (g -strip). For an integer $g \geq 1$, the g -strip with base bg is the interval of columns $[bg, (b+1)g)$, for $b = 0, 1, 2, \dots$. We call them simply *strips* when g is clear from context. A g -column is a column that is a multiple of g .

Definition 4.2 (g -sparse restriction). For an integer $g \geq 1$, a restriction of the input bit-vectors $\mathbf{x}$ and $\mathbf{y}$ is g -sparse if every partial product $t_{i,j}$ not forced to 0 lies in a g -column (recall Definition 2.4) and each g -column contains strictly fewer than 2^{g-1} such partial products.

Definition 4.3 (Strip-local). An exact multiplier encoding (Definition 2.3) is *strip-local* if its clauses have constant width and, under every g -sparse restriction of the input bit-vectors, every wire is a Boolean function of the partial products belonging to a single strip.

Remark 4.4. In the definition of g -sparse restriction, we set the threshold to 2^{g-1} instead of 2^g because Wallace-tree multipliers would not be strip-local with the latter threshold.

Observation 4.5 (Strips count their partial products). *Fix an exact multiplier encoding, an integer $g \geq 1$, and a g -sparse restriction of the input bit-vectors. Then, under every assignment to the unset inputs, the g output bits belonging to the strip with base bg compute, in binary, the number of partial products in the base column bg that evaluate to 1.*

Proposition 4.6. *The array multiplier encoding (Figure 1) is strip-local.*

Proof. Each variable of an array multiplier is determined by at most three other variables, so its clauses have width at most four.

Fix $g \geq 1$ and a g -sparse restriction. In an array multiplier, the only wires crossing a column boundary are carries (Figure 1), so a wire in the strip with base bg can depend on lower strips only through the carries entering column bg .

It therefore suffices to show that every carry feeding into a g -strip is the constant 0. This is trivial for the strip at base 0. Assume the inductive hypothesis that every carry feeding into the g -strip with base bg is zero. Then, since a carry into column $(b+1)g$ has weight $2^{(b+1)g}$, it can only be 1 if the base column bg of the previous strip contained at least 2^g nonzero partial products, contradicting the g -sparsity of the restriction. $\square$

Proposition 4.7. *The standard Wallace-tree multiplier encoding (Definition A.2), where the final two rows are summed by a carry-lookahead adder (Definition A.1), is strip-local.*

The proof is nontrivial due to the final carry-lookahead adder. Appendix A gives the proof.

5 Perfect matching and its star-local extension

The source of hardness for our resolution lower bound is the sparse perfect-matching principle studied in [ISS15, IOSS16]. In general, a perfect-matching principle asserts that a given graph has no perfect matching. Its CNF encoding negates this: it has one variable per edge and requires that every vertex is incident to exactly one selected edge. When the underlying graph is bipartite and has a different number of vertices on the two sides, this CNF is unsatisfiable.

The best-known special case is the pigeonhole principle, which is the perfect-matching principle of the complete bipartite graph $K_{m+1,m}$. Haken showed that the pigeonhole principle is hard for resolution [Hak85]. However, this is an inconvenient source of hardness for our reduction due to the high degree of its vertices. To absorb the auxiliary variables of the multiplier encodings, our reduction extends the formula with a new variable for every Boolean function of the edges at a vertex. For $K_{m+1,m}$, this introduces a doubly exponential number of new variables, so the Ben-Sasson–Wigderson size–width tradeoff (Theorem 2.1) does not give a useful size lower bound. On the other hand, for bounded-degree graphs, this extension only adds a constant number of variables and clauses per vertex, so the width lower bound of Itsykson, Slabodkin, and Sokolov [ISS15] implies an exponential size lower bound for the extension.

Definition 5.1 (Perfect-matching CNF). For a bipartite graph $G = (U, V, E)$, introduce a variable X_e for each edge $e \in E$. We interpret $X_e = 1$ to mean that the edge e is selected. The *exact-one perfect-matching CNF* is

$$\text{PM}(G) = \bigwedge_{w \in U \cup V} \text{ExactOne}(X_{E(w)}),$$

where $\text{ExactOne}(X_{E(w)})$ consists of the clauses

$$\bigvee_{e \in E(w)} X_e \quad \text{and} \quad \neg X_e \vee \neg X_f \quad (e, f \in E(w), e \neq f).$$

For the remainder of the paper, fix an integer m , a constant Δ independent of m , and a bipartite graph

$$H = (U, V, E), \quad |U| = m + 1, \quad |V| = m,$$

of maximum degree at most Δ and with no isolated vertices. We use a robust form of the hardness result of Itsykson, Slabodkin, and Sokolov [ISS15] for $\text{PM}(H)$, based on star-local extension variables in the style of the functional encoding of [ABRW04].

Definition 5.2 (Star-local extension). For each vertex w and Boolean function $f : \{0, 1\}^{E(w)} \rightarrow \{0, 1\}$, introduce an extension variable $Z_{w,f}$, defined by clauses $\text{Def}(Z_{w,f})$ over variables $\{Z_{w,f}\} \cup$

$\{X_e : e \in E(w)\}$ expressing that $Z_{w,f} = f((X_e)_{e \in E(w)})$. The *star-local extension* of the exact-one perfect-matching principle is

$$\text{SPM}(H) = \bigwedge_{w \in U \cup V} \left(\text{ExactOne}(X_{E(w)}) \wedge \bigwedge_{f: \{0,1\}^{E(w)} \rightarrow \{0,1\}} \text{Def}(Z_{w,f}) \right).$$

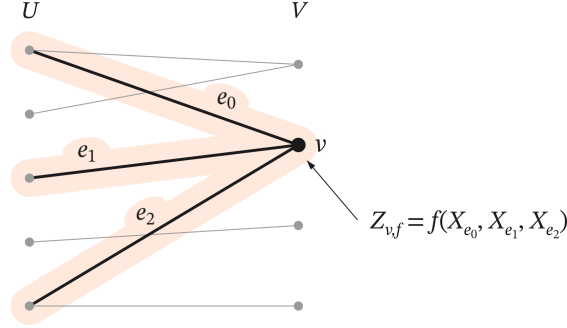


Figure 3: A star-local extension variable. The vertex v has star $E(v) = \{e_0, e_1, e_2\}$ (shaded), and for each Boolean function f on this star we introduce one variable $Z_{v,f}$ defined to equal $f(X_{e_0}, X_{e_1}, X_{e_2})$.

Figure 3 illustrates the extension at a single vertex. Every clause of $\text{SPM}(H)$ has width at most $\Delta + 1$. Because Δ is constant, each star has only a constant number of Boolean functions, so $\text{SPM}(H)$ still has $O(m)$ variables and $O(m)$ clauses.

Definition 5.3 (Boundary expander). Let $G = (U, V, E)$ be a bipartite graph. For $S \subseteq U$, the *boundary* $\delta(S) \subseteq V$ is the set of vertices of V with exactly one neighbor in S . The graph G is an (r, Γ) -*boundary expander* if

$$|\delta(S)| \geq \Gamma \cdot |S| \quad \text{for every } S \subseteq U \text{ with } |S| \leq r.$$

Theorem 5.4 ([ISS15, IOSS16]). *There are absolute constants $\Delta \in \mathbb{N}$, $\Gamma \geq 1$, and $\varepsilon > 0$ such that for every sufficiently large m there is a bipartite graph $H = (U, V, E)$ with $|U| = m + 1$ and $|V| = m$, of maximum degree at most Δ , that is an $(\varepsilon m, \Gamma)$ -boundary expander. Furthermore, H has a matching covering V .*

The following lemma is straightforward, though to our knowledge it has not explicitly appeared in the literature. A close analogue appears as Theorem 13(ii) of Atserias and Bonet [AB04], but they only considered extension variables representing conjunctions, rather than arbitrary Boolean functions. The proof itself is essentially the same as the proof of Lemma 4.3 given by Ben-Sasson and Nordström in [BN11], although the statement of the lemma below is different from what they conclude.

Lemma 5.5 (Width transfer). *Let Φ be an unsatisfiable CNF and F a set of Boolean functions. Let*

$$\Phi^{\text{ext}} = \Phi \wedge \bigwedge_{f \in F} \text{Def}(Z_f)$$

be an extension of Φ with extension variables Z_f , where $\text{Def}(Z_f)$ is a CNF expressing the constraint $Z_f = f(\mathbf{x}_f)$. If each tuple $\mathbf{x}_f$ consists of at most Δ variables of Φ , then every resolution refutation of Φ^{ext} has width at least $\frac{w(\Phi \vdash \perp)}{2\Delta}$.

Proof. Let π^{ext} be a resolution refutation of Φ^{ext} of width w . We obtain a refutation π of the CNF Φ of width at most $2\Delta w$ as follows. In each clause of π^{ext} , replace each extension variable Z_f by the Boolean function $f(\mathbf{x}_f)$. This turns each clause into a disjunction of at most w Boolean functions of at most Δ variables of Φ . Then replace each of these disjunctions by an equivalent CNF over its at most Δw variables.

Since resolution is sound, in each step of π^{ext} , the CNF of its resolvent is implied by the CNFs of its two premises. We obtain the refutation π by using implicational completeness to fill in the intermediate resolution steps between these CNFs. As the two premise CNFs, together, have at most $2\Delta w$ variables, these intermediate steps have width at most $2\Delta w$. $\square$

Theorem 5.6 (The star-local extension remains hard). *Let H be a bipartite expander given by Theorem 5.4. Then $\text{SPM}(H)$ requires resolution refutations of size $2^{\Omega(m)}$.*

Proof. Theorem 1.1 of the full version of [ISS15] shows a resolution width lower bound for $\text{PM}(H)$ of $\Omega(m)$. The maximum degree Δ of the graph H is constant, so Lemma 5.5 transfers this to a width lower bound for $\text{SPM}(H)$ of $\Omega(m)$. Since $\text{SPM}(H)$ has $O(m)$ variables and constant width, the size–width tradeoff (Theorem 2.1) gives a size lower bound of $2^{\Omega(m)}$. $\square$

Our reduction will also need the following observation, which follows immediately from the implicational completeness of resolution.

Lemma 5.7 (Local consequences have constant-size derivations). *For each vertex $w \in U \cup V$, let F_w denote the conjunction of the clauses of $\text{SPM}(H)$ associated with w ,*

$$F_w = \text{ExactOne}(X_{E(w)}) \wedge \bigwedge_{f: \{0,1\}^{E(w)} \rightarrow \{0,1\}} \text{Def}(Z_{w,f}),$$

so that $\text{SPM}(H) = \bigwedge_{w \in U \cup V} F_w$.

Let $W \subseteq U \cup V$. If a clause C is implied by $\bigwedge_{w \in W} F_w$, then either C is a tautology, or some subclause $C' \subseteq C$ has a resolution derivation from $\bigwedge_{w \in W} F_w$ whose size depends only on $|W|$ and on Δ .

6 Overview of the reduction

Throughout the reduction, we assume that Assoc_n uses a strip-local multiplier encoding Mul_n (Definition 4.3). At a high level, the reduction from $\text{SPM}(H)$ to Assoc_n works as follows. We begin by constructing a restriction ρ of Assoc_n such that the remaining clauses $\text{Assoc}_n \upharpoonright_\rho$ form a circuit encoding of the perfect-matching constraints. Because this encoding contains circuit variables that are not present in the perfect-matching formula $\text{PM}(H)$, the original perfect-matching lower bound does not immediately transfer to this restricted formula $\text{Assoc}_n \upharpoonright_\rho$. This is why we introduced the star-local extension $\text{SPM}(H)$. Each clause of the restricted formula $\text{Assoc}_n \upharpoonright_\rho$ has a constant-size resolution derivation from $\text{SPM}(H)$ (after substituting star-local extension variables in place of the clause’s wire variables). Therefore a resolution refutation of Assoc_n yields a resolution refutation of $\text{SPM}(H)$ with only a constant-factor increase in size. The exponential lower bound for $\text{SPM}(H)$ (Theorem 5.6) thereby transfers to Assoc_n . Figure 4 shows the shape of the composed refutation.

6.1 Reduction of star-local perfect matching to Assoc_n

Suppose that π is a resolution refutation of Assoc_n of size s . We convert π into a refutation of $\text{SPM}(H)$ of size $O(s)$ in three steps.

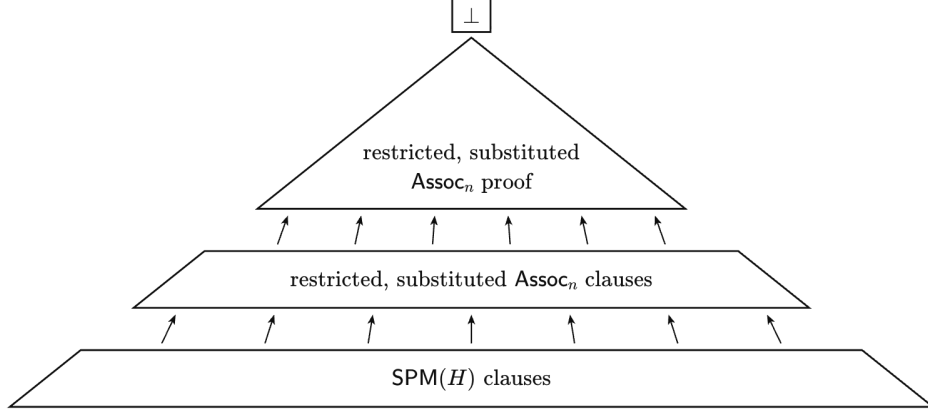


Figure 4: The refutation of $\text{SPM}(H)$, the star-local extension of perfect matching. This refutation uses $\text{SPM}(H)$ to derive a subclause of each clause of the restricted, substituted Assoc_n in a constant number of steps. The restricted, substituted refutation of Assoc_n then derives $\perp$ from these clauses.

- **(Restrict)** We construct a restriction ρ that leaves one input bit of $\mathbf{y}$ unset for each edge $e \in E$ and identifies it with the perfect-matching edge variable X_e . It sets the remaining input bits of $\mathbf{x}, \mathbf{y}, \mathbf{z}$ to constants 0 or 1. The restriction ρ also sets some of the output bits of the multipliers $\mathbf{xy}$ and $\mathbf{yz}$, and it sets one of the miter variables $e_K = 1$ at a fixed position K , satisfying the long miter clause $e_0 \vee \dots \vee e_{4n-1}$.

Under the restriction ρ , the two inner multipliers of Assoc_n become encodings, with auxiliary variables, of the perfect-matching constraints on the edge variables X_e (Lemma 8.5). More precisely, under ρ every wire computes a Boolean function of the edge variables, so an assignment α to the edge variables X_e propagates to an assignment $\bar{\alpha}$ of all remaining variables (Definition 8.1). The propagated assignment $\bar{\alpha}$ satisfies the restricted encoding $\mathbf{xy} \upharpoonright_\rho$ if and only if the original edge variable assignment α satisfies $\text{ExactOne}(X_{E(v)})$ for every $v \in V$. Similarly, $\bar{\alpha}$ satisfies $\mathbf{yz} \upharpoonright_\rho$ if and only if α satisfies $\text{ExactOne}(X_{E(u)})$ for every $u \in U$ (Lemma 8.5).

Applying the restriction ρ to the proof π yields a refutation $\pi \upharpoonright_\rho$ of the restricted associativity formula $\text{Assoc}_n \upharpoonright_\rho$.

The restricted formula $\text{Assoc}_n \upharpoonright_\rho$ contains no multiplier input variables other than the edge variables X_e . Its remaining variables are wires, which the next step eliminates.

- **(Substitute)** This restriction ρ will also ensure that every wire computes some function $f(X_{E(w)})$ of the edge variables of a single star $E(w)$, $w \in U \cup V$ (Lemma 8.4). We can therefore substitute each wire throughout the refutation $\pi \upharpoonright_\rho$ by the corresponding extension variable $Z_{w,f}$. After this substitution, every clause C of the restricted associativity formula $\text{Assoc}_n \upharpoonright_\rho$ is implied by the clauses of $\text{SPM}(H)$ associated with a set of stars covering C 's variables (Lemma 8.6). Since each clause C of $\text{Assoc}_n \upharpoonright_\rho$ has constant width, its variables can be covered by a constant number of stars.
- **(Compose)** Finally, we obtain a refutation of the star-local extension $\text{SPM}(H)$ as follows. Using Lemma 5.7, we derive a subclause of each restricted, substituted Assoc_n clause from the clauses of $\text{SPM}(H)$ in only a constant number of steps. Then we apply the restricted, substituted refutation of Assoc_n to refute the derived clauses.

6.2 Outline of the restriction ρ

We will define a g -sparse restriction ρ in terms of three maps. We map each vertex $u \in U$ to an $\mathbf{x}$ -bit-position $i(u)$, each edge $e \in E$ to a $\mathbf{y}$ -bit-position $j(e)$, and each vertex $v \in V$ to a $\mathbf{z}$ -bit-position $k(v)$. For each vertex $u \in U$ and $v \in V$, the restriction ρ sets $x_{i(u)} = 1$ and $z_{k(v)} = 1$. For each edge $e \in E$, the restriction ρ leaves $y_{j(e)}$ unrestricted and identifies it with the edge variable X_e . It restricts all remaining bits of $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$ to 0. Additionally, ρ restricts some of the output bits of the inner multipliers $\mathbf{xy}$ and $\mathbf{yz}$, together with one miter variable, as we will specify below.

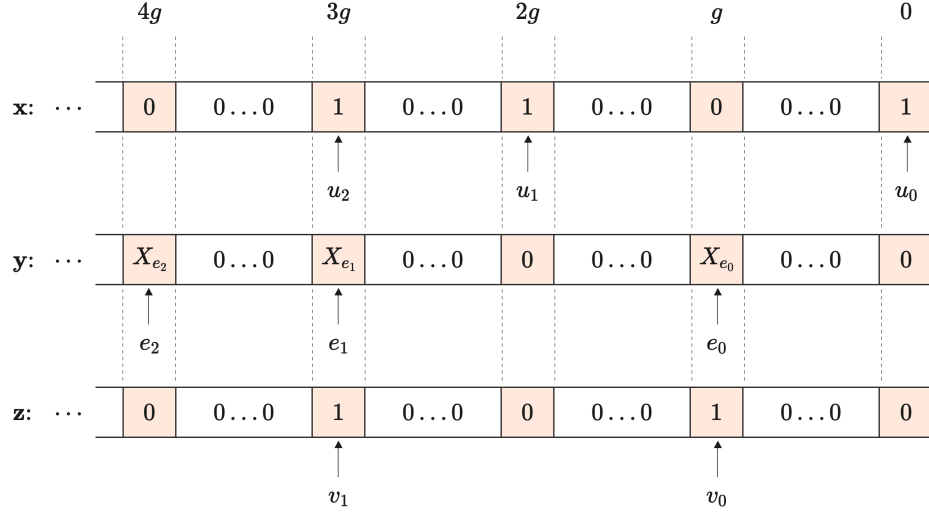


Figure 5: The restricted input bit-vectors. Each vertex $u \in U$ maps to a 1 in $\mathbf{x}$ at position $i(u)$, each vertex $v \in V$ maps to a 1 in $\mathbf{z}$ at position $k(v)$, and each edge $e \in E$ maps to an unrestricted variable $y_{j(e)}$ in $\mathbf{y}$ that we identify with the edge variable X_e . Each position is a multiple of g , and every other bit is fixed to 0.

Lemma 7.3 constructs the maps $i(u)$, $j(e)$, $k(v)$ with special collision properties. In this outline we will use the fact that for an edge $e = (u_e, v_e)$, this construction sets

$$j(e) = K - i(u_e) - k(v_e),$$

where K is a fixed multiple of g . Under ρ , the four multipliers behave as follows.

- **(g -columns)** For each vertex and edge, the values $i(u), j(e), k(v)$ are multiples of the strip width $g = \lceil \log(m+2) \rceil + 2$ (Lemma 7.3). This ensures that the nonzero partial products in each of the four multipliers occur only in g -columns (Definition 4.1). Figure 5 shows the restricted input vectors $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$. Moreover, every g -column will contain fewer than 2^{g-1} of these nonzero partial products, making ρ a g -sparse restriction of each of the four multipliers (Lemma 7.5). By strip-locality, every wire is a Boolean function of the partial products appearing in a single g -strip.
- **(Inner multipliers)** Each vertex $v \in V$ is associated with a unique g -column of the multiplier $\mathbf{xy}$, whose index is $\text{col}_{\mathbf{xy}}(v) := K - k(v)$. This column contains the partial products that, under ρ , equal the variables X_e for edges e of the star $E(v)$. So the effect of the multiplier circuit $\mathbf{xy}$ is to constrain the output of the g -strip based at $\text{col}_{\mathbf{xy}}(v)$ to equal the sum $\sum_{e \in E(v)} X_e$ represented in binary.

For each $v \in V$, the restriction ρ fixes the output of the g -strip based at $\text{col}_{\mathbf{xy}}(v)$ to the binary representation of 1. This enforces $\sum_{e \in E(v)} X_e = 1$, which is equivalent to the constraint $\text{ExactOne}(X_{E(v)})$. Figure 6 illustrates the effect of the restriction ρ in the multiplier $\mathbf{xy}$.

Symmetrically, each $u \in U$ is associated with a unique g -column of the multiplier $\mathbf{yz}$, whose index is $\text{col}_{\mathbf{yz}}(u) := K - i(u)$, and which contains the variables X_e for edges e of the star $E(u)$. The restriction ρ fixes the output of the g -strip based at $\text{col}_{\mathbf{yz}}(u)$ to the binary representation of 1, enforcing $\text{ExactOne}(X_{E(u)})$.

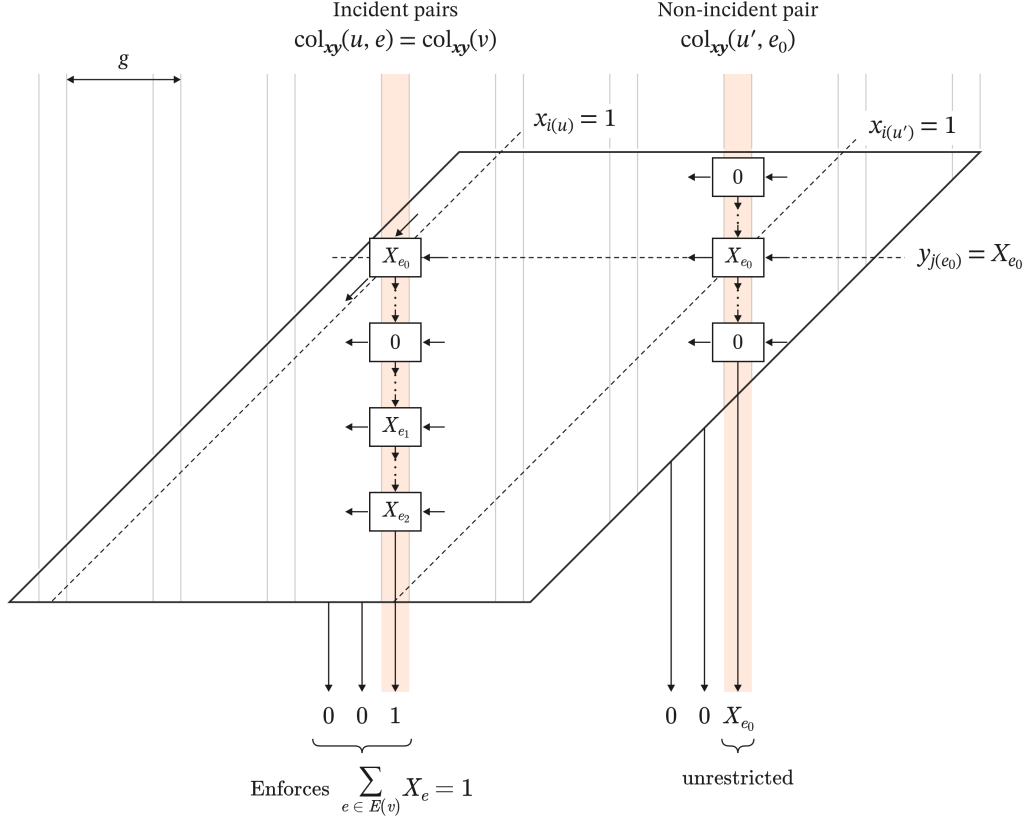


Figure 6: An inner array multiplier $\mathbf{xy}$ under the restriction ρ . Incident pairs (u_e, e) with $e \in E(v)$ map to the common column $\text{col}_{\mathbf{xy}}(u_e, e) = \text{col}_{\mathbf{xy}}(v)$, so this column contains the partial products X_e for all edges $e \in E(v)$. The restriction ρ fixes the output of this column's g -strip to the binary representation of 1, enforcing $\sum_{e \in E(v)} X_e = 1$. Each non-incident pair (u, e) maps to its own column $\text{col}_{\mathbf{xy}}(u, e)$, so the output of this column is X_e .

- **(Outer multipliers)** Column K of the outer multiplier $(\mathbf{xy})\mathbf{z}$ contains exactly one 1 for each $v \in V$, as shown in Figure 7: the restriction ρ sets the output bit of $\mathbf{xy}$ at index $\text{col}_{\mathbf{xy}}(v) = K - k(v)$ to 1, and multiplying this output by $z_{k(v)} = 1$ contributes a 1 to column K . The remaining partial products in column K are 0 since ρ restricts every other bit of $\mathbf{z}$ to 0.

Therefore column K of $(\mathbf{xy})\mathbf{z}$ contains exactly $|V| = m$ ones, so the output of the g -strip based at K is forced to the binary representation of m (Observation 4.5). Symmetrically, column K of the outer multiplier $\mathbf{x}(\mathbf{yz})$ contains a 1 for each $u \in U$, so the output of the g -strip based at K is forced to the binary representation of $m + 1$. This means that the outer multipliers disagree on their K -th output bit, so ρ sets the miter variable $e_K = 1$ to satisfy the long miter clause $e_0 \vee \dots \vee e_{4n-1}$. This completes the description of ρ .

- **(Other g -columns)** In all four multipliers, each remaining g -column not covered above contains edge variables from at most one star (Figures 6 and 8). Therefore every wire in the corresponding g -strips is a Boolean function of a single star.

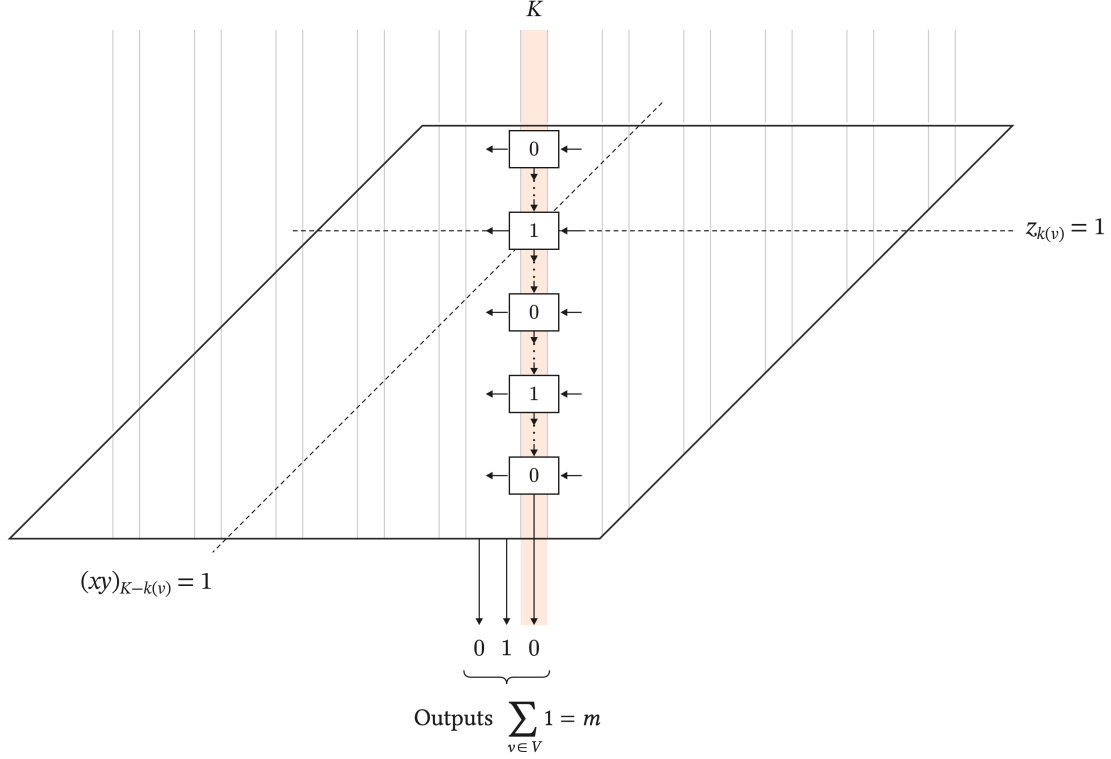


Figure 7: An outer array multiplier $(\mathbf{xy})\mathbf{z}$ under the restriction ρ . Column K contains a constant 1 for each $v \in V$, which comes from the output bit of $\mathbf{xy}$ at $\text{col}_{\mathbf{xy}}(v)$ multiplied by $z_{k(v)} = 1$. Every other partial product in the column is 0, so the g -strip based at K outputs the binary representation of m . In the other outer multiplier $\mathbf{x}(\mathbf{yz})$, the same g -strip outputs $m + 1$.

7 Construction of the restriction ρ

In this section we build the index maps embedding the bipartite graph $H = (U, V, E)$ into the input bit-vectors $\mathbf{x}, \mathbf{y}, \mathbf{z}$. We will then use these maps to define our g -sparse restriction ρ .

7.1 Construction of the index maps $i(u), j(e), k(v)$

We build three maps: $i : U \rightarrow \mathbb{N}$ for $\mathbf{x}$ -bit-positions, $j : E \rightarrow \mathbb{N}$ for $\mathbf{y}$ -bit-positions, and $k : V \rightarrow \mathbb{N}$ for $\mathbf{z}$ -bit-positions. Recall from Section 6.2 that the restriction ρ will set $x_{i(u)} = 1$ and $z_{k(v)} = 1$, identify each $y_{j(e)}$ with the edge variable X_e , and restrict all remaining bits of $\mathbf{x}, \mathbf{y}$, and $\mathbf{z}$ to 0.

Thus $u \in U$, $e \in E$, and $v \in V$ index bits of $\mathbf{x}, \mathbf{y}$, and $\mathbf{z}$ respectively. Pairs and triples of them point to partial products in the multipliers: the pair (u, e) points to the partial product $x_{i(u)}y_{j(e)}$ of the inner multiplier $\mathbf{xy}$, in column $i(u) + j(e)$, the pair (e, v) points to $y_{j(e)}z_{k(v)}$ of $\mathbf{yz}$, in column $j(e) + k(v)$, and the triple (u, e, v) points to an outer partial product in each of $(\mathbf{xy})\mathbf{z}$ and $\mathbf{x}(\mathbf{yz})$, both in the same column $i(u) + j(e) + k(v)$. We call (u, e) , (e, v) , and (u, e, v) *incident* when the vertices are endpoints of e . Accordingly, we define maps from pairs and triples to columns.

Definition 7.1 (Columns). For maps $i : U \rightarrow \mathbb{N}$, $j : E \rightarrow \mathbb{N}$, and $k : V \rightarrow \mathbb{N}$ and $u \in U$, $e \in E$, and $v \in V$, let

$$\begin{aligned} \text{col}_{\mathbf{xy}}(u, e) &:= i(u) + j(e), & \text{col}_{\mathbf{yz}}(e, v) &:= j(e) + k(v), \\ \text{col}_{(\mathbf{xy})\mathbf{z}}(u, e, v) &:= i(u) + j(e) + k(v), & \text{col}_{\mathbf{x}(\mathbf{yz})}(u, e, v) &:= i(u) + j(e) + k(v). \end{aligned}$$

We build the maps $i(u)$, $j(e)$, $k(v)$ from a Golomb ruler, which prevents unintended collisions when we map vertices and edges to g -strips.

Lemma 7.2 (Directed Golomb ruler). *For every $m \geq 1$, set $M = 32m^2$. There are $m + 1$ integers $s_0, \dots, s_m \in [0, M)$ called marks such that*

$$s_a - s_b = s_c - s_d \neq 0 \implies (a, b) = (c, d).$$

In other words, every nonzero directed difference is produced by a unique ordered pair of marks.

Proof. Use the Erdős–Turán construction [ET41]. Choose an odd prime $m + 1 < p < 2(m + 1)$ and set

$$s_r = 2pr + (r^2 \bmod p), \quad 0 \leq r < p.$$

Since $s_r < 2p^2 < 8(m + 1)^2 \leq 32m^2 = M$, every mark lies in the required range. Take the first $m + 1$ marks $s_0, \dots, s_m$. Put $\tau_r := r^2 \bmod p \in \{0, \dots, p - 1\}$. If $s_a - s_b = s_c - s_d \neq 0$, then

$$2p((a - b) - (c - d)) = (\tau_c - \tau_d) - (\tau_a - \tau_b).$$

The right side has absolute value $< 2p$, while the left side is a multiple of $2p$, so $a - b = c - d =: h$. Reducing the original equality modulo p gives $h(a + b) \equiv h(c + d)$. Since the difference is nonzero, $h \neq 0$ and $|h| < p$, so cancel h to get $a + b \equiv c + d \pmod{p}$. Adding and subtracting this congruence with $a - b = c - d$ gives $2a \equiv 2c$ and $2b \equiv 2d \pmod{p}$. Because p is odd, 2 is invertible, hence $a \equiv c$ and $b \equiv d \pmod{p}$. As $a, b, c, d \in \{0, \dots, p - 1\}$, we have $a = c$ and $b = d$. $\square$

Lemma 7.3 (Construction of index maps). *Let $H = (U, V, E)$ be a bipartite graph with $|U| = m + 1$ and $|V| = m$, and set*

$$g = \lceil \log(m + 2) \rceil + 2.$$

Then there is an integer $n_0 = O(m^4 \log m)$ as well as maps

$$i : U \rightarrow \mathbb{N}, \quad j : E \rightarrow \mathbb{N}, \quad k : V \rightarrow \mathbb{N}$$

with the following properties.

(i) **Mapping pattern.** *The maps i , j , and k map injectively to multiples of g . Moreover, $i(u) + j(e) + k(v) < n_0$ for all $u \in U$, $e \in E$, and $v \in V$.*

(ii) **Inner collision pattern.**

(a) *For each vertex $v \in V$ the incident pairs (u_e, e) with edge $e \in E(v)$ map to a common column $\text{col}_{\mathbf{xy}}(u_e, e)$, denoted $\text{col}_{\mathbf{xy}}(v)$. Distinct vertices $v \in V$ map to distinct columns $\text{col}_{\mathbf{xy}}(v)$. Each non-incident pair (u, e) occupies its own column $\text{col}_{\mathbf{xy}}(u, e)$.*

(b) *For each vertex $u \in U$ the incident pairs (e, v_e) with edge $e \in E(u)$ map to a common column $\text{col}_{\mathbf{yz}}(e, v_e)$, denoted $\text{col}_{\mathbf{yz}}(u)$. Distinct vertices in U map to distinct columns $\text{col}_{\mathbf{yz}}(u)$. Each non-incident pair (e, v) occupies its own column $\text{col}_{\mathbf{yz}}(e, v)$.*

(iii) **Outer collision pattern.** *Define the difference pair of a triple $(u, e, v) \in U \times E \times V$ as*

$$(i(u) - i(u_e), k(v) - k(v_e)).$$

The column $\text{col}_{(\mathbf{xy})\mathbf{z}}(u, e, v)$ depends only on the difference pair of the triple, and distinct difference pairs map to distinct columns. Furthermore:

- (a) The triples with difference pair $(0, 0)$ are exactly the incident triples (u_e, e, v_e) , $e \in E$. We denote their common column by K .
- (b) Each nonzero difference determines its ordered pair of vertices: if $i(u) - i(u_e) = i(u') - i(u_{e'}) \neq 0$ then $(u, u_e) = (u', u_{e'})$, and if $k(v) - k(v_e) = k(v') - k(v_{e'}) \neq 0$ then $(v, v_e) = (v', v_{e'})$.

Before giving the proof, we describe what properties (ii) and (iii) will mean for the g -strips of the four multipliers under ρ . Property (ii) ensures that in the inner multipliers every g -column holds either the edge variables of a single star $E(u)$ or $E(v)$, or at most one edge variable X_e (Figure 6). Since ρ will fix the output of each single-star strip to the binary representation of 1 (Section 6.2), the inner multipliers then enforce exactly the constraints $\text{ExactOne}(X_{E(v)})$ and $\text{ExactOne}(X_{E(u)})$, for $v \in V$ and $u \in U$.

Property (iii) assigns each difference pair to a unique g -column at the same position in both outer multipliers. This serves two purposes. First, by (iii)(a) the partial products lying in the g -column for the difference pair $(0, 0)$ are exactly those corresponding to incident triples (u_e, e, v_e) . As described in Section 6.2, the restriction ρ forces these partial products to constants, which then forces the strip at K to output m in $(\mathbf{xy})\mathbf{z}$ and $m + 1$ in $\mathbf{x}(\mathbf{yz})$ (Figure 7). Second, (iii)(b) ensures that each remaining outer multiplier g -column contains edge variables from at most one star (Figure 8). For example, consider the case of a g -column corresponding to the difference pair $(d, 0)$ with $d \neq 0$. The difference d is produced by a unique ordered pair (u, u_e) with $d = i(u) - i(u_e)$, so only edge variables X_e for edges $e \in E(u_e)$ can appear in this g -column. The same reasoning applies to every g -column corresponding to a non- $(0, 0)$ difference pair.

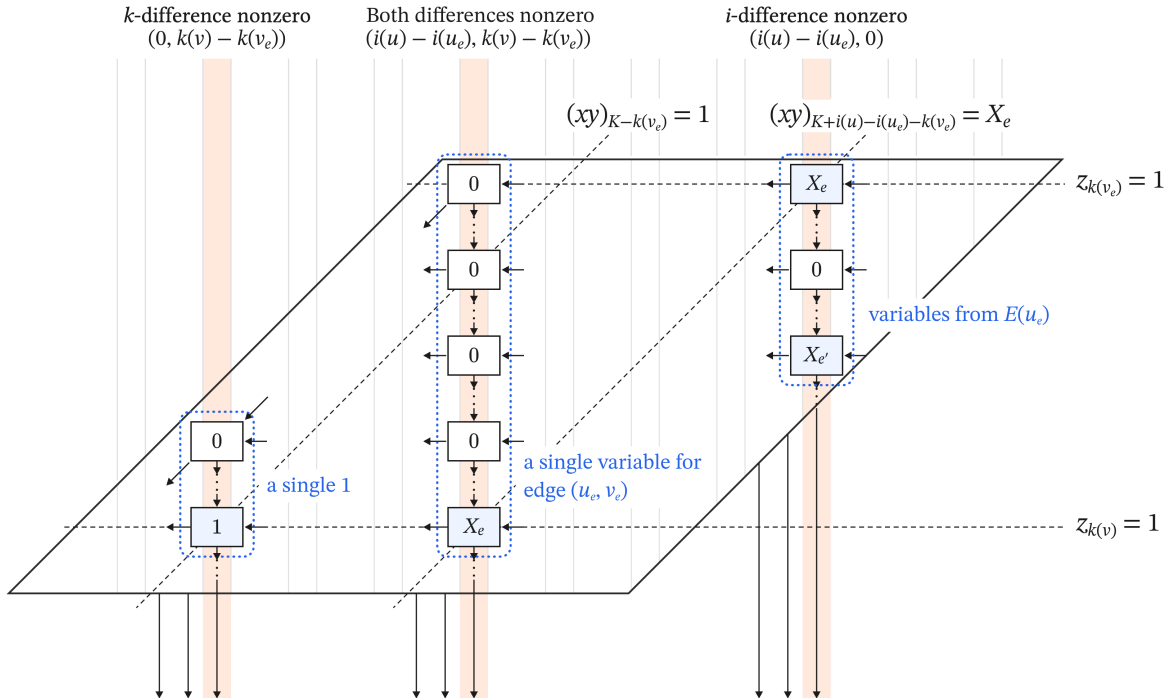


Figure 8: Aside from column K , there are three kinds of g -columns of the outer multiplier $(\mathbf{xy})\mathbf{z}$ under the restriction ρ . In each case, the wires inside the corresponding g -strip are functions of the Boolean variables from a single star.

Proof. With $M = 32m^2$, let $s_0, \dots, s_m \in [0, M]$ be the marks given by Lemma 7.2. Choose injections of U and V separately into these marks, and write s_u and s_v for the marks assigned to $u \in U$ and $v \in V$. To obtain property (iii), we want the outer column

$$\text{col}_{(\mathbf{xy})\mathbf{z}}(u, e, v) = i(u) + j(e) + k(v)$$

to be uniquely determined by the two directed differences $s_u - s_{u_e}$ and $s_v - s_{v_e}$. Since both differences have magnitude $< M$, set

$$B = 2M + 1.$$

Then the two-digit base- B value $a + Bb$, for $|a|, |b| < M$, uniquely determines its digits. We refer to this as *base- B uniqueness*. We therefore scale the U -marks by g and the V -marks by gB , and build $j(e)$ so that $\text{col}_{(\mathbf{xy})\mathbf{z}}$ is the same column for all incident triples (u_e, e, v_e) . To this end, set

$$i(u) = gs_u, \quad k(v) = gBs_v, \quad j(e) = K - i(u_e) - k(v_e) \quad (1)$$

where we choose the constant $K = 2g(B+1)M$ to make $j(e)$ non-negative. Set $n_0 := 4K$. With these parameters, $n_0 = O(m^4 \log m)$.

Proof of (i). It is clear from the construction that each $i(u), j(e), k(v)$ is a multiple of g . The maps i and k are injective because the marks s_u and s_v are assigned injectively. For the map j , since

$$j(e) = K - g(s_{u_e} + Bs_{v_e}),$$

base- B uniqueness shows that $j(e) = j(e')$ implies $u_e = u_{e'}$ and $v_e = v_{e'}$, hence $e = e'$. Finally, $i(u) + j(e) + k(v) = K + g(s_u - s_{u_e}) + gB(s_v - s_{v_e})$ exceeds K by less than $g(B+1)M = K/2$, so it is less than $3K/2 < n_0$.

Proof of (ii). For the maps $i(u)$ and $j(e)$ as defined in (1), we have

$$\text{col}_{\mathbf{xy}}(u, e) = i(u) + j(e) = K - k(v_e) + g(s_u - s_{u_e}). \quad (2)$$

Thus, for each $v \in V$, the incident pairs (u_e, e) with $e \in E(v)$ share the column $\text{col}_{\mathbf{xy}}(v) := K - k(v)$, and these columns are distinct for distinct v because k is injective. Each non-incident pair (u, e) occupies its own column because of base- B uniqueness and the fact that the difference $s_u - s_{u_e}$ is attained by a unique pair (u, u_e) (Lemma 7.2). This proves (ii)(a).

Symmetrically,

$$\text{col}_{\mathbf{yz}}(e, v) = K - i(u_e) + gB(s_v - s_{v_e}), \quad (3)$$

and the same argument proves (ii)(b).

Proof of (iii). For the maps $i(u), j(e), k(v)$ defined in (1), we have

$$\begin{aligned} \text{col}_{(\mathbf{xy})\mathbf{z}}(u, e, v) &= K + (i(u) - i(u_e)) + (k(v) - k(v_e)) \\ &= K + g(s_u - s_{u_e}) + gB(s_v - s_{v_e}). \end{aligned}$$

The first line shows that the column depends only on the difference pair of the triple. The second line shows, by base- B uniqueness, that two columns $\text{col}_{(\mathbf{xy})\mathbf{z}}(u, e, v)$ and $\text{col}_{(\mathbf{xy})\mathbf{z}}(u', e', v')$ are equal if and only if they agree on both differences: $s_u - s_{u_e} = s_{u'} - s_{u_{e'}}$ and $s_v - s_{v_e} = s_{v'} - s_{v_{e'}}$. Hence, distinct difference pairs map to distinct columns.

For (a), a triple has difference pair $(0, 0)$ if and only if $u = u_e$ and $v = v_e$, that is, the triple is incident.

For (b), suppose that $i(u) - i(u_e) = i(u') - i(u_{e'}) \neq 0$. Then $s_u - s_{u_e} = s_{u'} - s_{u_{e'}} \neq 0$, so by Lemma 7.2 we must have $u = u'$ and $u_e = u_{e'}$. The case of a nonzero k -difference is symmetric. $\square$

The rest of the argument uses only the three properties of Lemma 7.3.

7.2 The restriction ρ

Using the maps $i(u), j(e), k(v)$ provided by Lemma 7.3, we now give the full description of the restriction ρ . We then prove that it is g -sparse, and that it forces the outputs of the outer multipliers to disagree at column K .

Definition 7.4 (The restriction ρ). Let the maps $i(u), j(e), k(v)$ be as in Lemma 7.3, let $g = \lceil \log(m+2) \rceil + 2$ be the strip width, and let K be the column given by Lemma 7.3(iii).

- (a) **(Inputs)** For each vertex $u \in U$ and $v \in V$, the restriction ρ sets $x_{i(u)} = 1$ and $z_{k(v)} = 1$. For each edge $e \in E$, ρ leaves the bit $y_{j(e)}$ unset and identifies it with the edge variable X_e ; it restricts all remaining bits of $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$ to 0.
- (b) **(Inner multiplier outputs)** Recall that Lemma 7.3(ii) provides well-defined and injective column maps $\text{col}_{\mathbf{xy}}(v)$ for $v \in V$ and $\text{col}_{\mathbf{yz}}(u)$ for $u \in U$. For each $v \in V$, the restriction ρ fixes the output of the g -strip of $\mathbf{xy}$ with base $\text{col}_{\mathbf{xy}}(v)$ to the binary representation of 1 (ρ sets the output bit in column $\text{col}_{\mathbf{xy}}(v)$ to 1 and the other $g-1$ output bits to 0), and for each $u \in U$, it fixes the output of the g -strip of $\mathbf{yz}$ with base $\text{col}_{\mathbf{yz}}(u)$ to the binary representation of 1. The restriction ρ leaves the output bit in column $\text{col}_{\mathbf{xy}}(u, e)$ unset for each non-incident pair (u, e) , and sets every other output bit of $\mathbf{xy}$ to 0. Symmetrically, ρ leaves the output bit in column $\text{col}_{\mathbf{yz}}(e, v)$ unset for each non-incident pair (e, v) , and sets every other output bit of $\mathbf{yz}$ to 0.
- (c) **(Miter)** The restriction ρ sets the miter variable $e_K = 1$.

Lemma 7.5 (ρ is a g -sparse restriction). *The following hold.*

- (i) *Every partial product of $\mathbf{xy}$ not forced to 0 lies in a g -column. Additionally, for each $v \in V$, the g -column $\text{col}_{\mathbf{xy}}(v)$ contains exactly the partial products*

$$x_{i(u_e)}y_{j(e)} = X_e, \quad e \in E(v),$$

and every other g -column contains at most one partial product not forced to 0; symmetrically for $\mathbf{yz}$ with the g -columns $\text{col}_{\mathbf{yz}}(u)$, $u \in U$.

- (ii) *For each of the four multipliers, ρ is a g -sparse restriction of its input bit-vectors.*

Proof. (i) Consider the inner multiplier $\mathbf{xy}$. A partial product $x_i y_j$ is not forced to 0 by the restriction ρ if and only if x_i is set to 1 and y_j is not set to 0, that is, $i = i(u)$ and $j = j(e)$ for some $u \in U$ and $e \in E$. The partial product then computes $x_{i(u)}y_{j(e)} = X_e$ and lies in column $\text{col}_{\mathbf{xy}}(u, e)$, which is a g -column by Lemma 7.3(i). By Lemma 7.3(ii)(a), the pairs (u, e) with $\text{col}_{\mathbf{xy}}(u, e) = \text{col}_{\mathbf{xy}}(v)$ are exactly the incident pairs (u_e, e) with $e \in E(v)$, while every non-incident pair occupies a g -column of its own. Hence the g -column $\text{col}_{\mathbf{xy}}(v)$ contains exactly the partial products X_e , $e \in E(v)$, and every other g -column contains at most one partial product X_e . The argument for $\mathbf{yz}$ is symmetric.

(ii) Since $2^{g-1} \geq 2(m+2) > m+1$, it suffices to show that the partial products of each multiplier not forced to 0 lie in g -columns, with at most $m+1$ such partial products per g -column. For the inner multipliers this follows from (i), since every star of H has at most $m+1$ edges.

Now consider the outer multiplier $(\mathbf{xy})\mathbf{z}$. The restriction ρ sets each of the bits in $\mathbf{z}$ to 0 except for the bits $z_{k(v)}$, and sets each of the bits in $\mathbf{xy}$ to 0 except for those at the columns $\text{col}_{\mathbf{xy}}(u, e)$. So every partial product of $(\mathbf{xy})\mathbf{z}$ not forced to 0 lies in a column of the form $\text{col}_{\mathbf{xy}}(u, e) + k(v) = \text{col}_{(\mathbf{xy})\mathbf{z}}(u, e, v)$. This is a g -column by Lemma 7.3(i). Within a given column of $(\mathbf{xy})\mathbf{z}$, distinct partial products have distinct factors $z_{k(v)}$ and hence correspond to distinct $v \in V$. So each

g -column contains at most $|V| = m$ partial products not forced to 0. The same holds for the g -columns of $\mathbf{x}(\mathbf{y}\mathbf{z})$ with the $m + 1$ vertices $u \in U$. $\square$

Lemma 7.6 (ρ forces disagreement). *Under the restriction ρ , the outer multiplier strips at base K are forced to output the binary representation of m in $(\mathbf{x}\mathbf{y})\mathbf{z}$ and of $m + 1$ in $\mathbf{x}(\mathbf{y}\mathbf{z})$. In particular, the output bits at column K disagree.*

Proof. We claim that the g -column K of the multiplier $(\mathbf{x}\mathbf{y})\mathbf{z}$ contains exactly m ones, one for each $v \in V$. Indeed, for each $v \in V$, the restriction ρ sets the output bit of $\mathbf{x}\mathbf{y}$ at $\text{col}_{\mathbf{x}\mathbf{y}}(v)$ and the input bit $z_{k(v)}$ to 1. Their product is a partial product lying in column $\text{col}_{\mathbf{x}\mathbf{y}}(v) + k(v) = K$, and by Lemma 7.3(ii),(iii) it is the only partial product in column K containing $z_{k(v)}$. Every other partial product in column K is forced to 0, since ρ restricts every other bit of $\mathbf{z}$ to 0. By Lemma 7.5(ii), ρ is a g -sparse restriction of the inputs of the multiplier $(\mathbf{x}\mathbf{y})\mathbf{z}$, hence Observation 4.5 applies and the strip at base K is forced to output the binary representation of m (see Figure 7). Symmetrically, column K of the multiplier $\mathbf{x}(\mathbf{y}\mathbf{z})$ contains a 1 for each $u \in U$, so its strip at base K is forced to output the binary representation of $m + 1$. Since m and $m + 1$ disagree in their least significant bit, the output bits at column K disagree. $\square$

8 Resolution lower bound for Assoc_n

With the restriction ρ constructed (Definition 7.4), we now carry out the substitute and compose steps outlined in Section 6.1 to prove the lower bound. Throughout this section, we assume that $n \geq n_0$, and that Assoc_n uses a strip-local multiplier encoding Mul_n .

For the following definitions, recall that under the restriction ρ , every variable of Assoc_n computes a Boolean function of edge variables X_e .

Definition 8.1 (Propagated assignment). For an assignment α to the edge variables X_e , the *propagated assignment* $\bar{\alpha}$ extends ρ by setting each variable of $\text{Assoc}_n \upharpoonright_\rho$ to the value of its Boolean function at α .

Definition 8.2 (Edge- and star-local). Under ρ , a circuit wire is *edge-local* if it is not constant and computes a Boolean function of a single edge variable X_e . It is *star-local at w* if it is neither constant nor edge-local and computes a Boolean function of the variables $\{X_e : e \in E(w)\}$.

Definition 8.3 (Support of a restricted clause). The *support* of a clause C of $\text{Assoc}_n \upharpoonright_\rho$ is the set $\text{supp}(C)$ of vertices $w \in U \cup V$ such that some variable of C depends on an edge variable of the star $E(w)$.

Lemma 8.4 (Locality). *Under the restriction ρ , every wire of the four multipliers and every miter variable is constant, edge-local, or star-local. Moreover, every wire of the inner multiplier $\mathbf{x}\mathbf{y}$ computes a Boolean function of a single star $E(v)$, $v \in V$, and every wire of $\mathbf{y}\mathbf{z}$ computes a Boolean function of a single star $E(u)$, $u \in U$.*

Proof. Since ρ is a g -sparse restriction of the inputs for each of the four multipliers (Lemma 7.5(ii)) and Mul_n is a strip-local encoding, under ρ every wire of these multipliers computes a Boolean function of the partial products within a single strip's base column. The statement then follows for the inner multipliers $\mathbf{x}\mathbf{y}$ and $\mathbf{y}\mathbf{z}$ by Lemma 7.5(i).

For the outer multipliers, each of their input bits is either constant or computes a single edge variable X_e . Consider, for example, the multiplier $(\mathbf{x}\mathbf{y})\mathbf{z}$. Its input bits $\mathbf{z}$ are all set to constants by ρ . In the other input bit-vector $(\mathbf{x}\mathbf{y})$, each unrestricted bit is the base output bit of an inner multiplier

strip holding a single partial product X_e (Definition 7.4(b), Lemma 7.5(i)). By Observation 4.5 this output bit computes X_e .

By Definition 7.4(a),(b), the partial products not forced to 0 in either outer multiplier are indexed by triples $(u, e, v) \in U \times E \times V$. By Lemma 7.3(iii), the column of such a partial product depends only on its difference pair $(i(u) - i(u_e), k(v) - k(v_e))$, and each nonzero difference is realized by a unique ordered pair of vertices from U or V . There are therefore four kinds of outer g -columns to check, according to which entries of the difference pair are 0 (Figures 7 and 8). It suffices to show that each kind of column only contains edge variables from at most one star.

- **Difference pair $(0, 0)$.** By Lemma 7.3(iii)(a), this is the g -column K , which contains only constant partial products.
- **Difference pair $(d, 0)$ with $d \neq 0$.** By Lemma 7.3(iii)(b), we have $d = i(u) - i(u_e)$ for a unique ordered pair (u, u_e) , so the triples mapping to this column are (u, e, v_e) with $e \in E(u_e)$. In the outer multiplier $(\mathbf{xy})\mathbf{z}$, this column therefore contains exactly the edge variables from the single star $E(u_e)$. In $\mathbf{x}(\mathbf{yz})$, the only partial product of this column not forced to 0 is the product of the input bit $x_{i(u)}$ with the output bit of $\mathbf{yz}$ at $\text{col}_{\mathbf{yz}}(u_e)$, and ρ sets both bits to 1.
- **Difference pair $(0, d)$ with $d \neq 0$.** Symmetrically, in the multiplier $\mathbf{x}(\mathbf{yz})$ this column contains exactly the edge variables from the single star $E(v_e)$, while in the multiplier $(\mathbf{xy})\mathbf{z}$ the only partial product not forced to 0 in this column is the constant 1.
- **Both differences nonzero.** By Lemma 7.3(iii)(b), the two differences determine ordered pairs (u, u') and (v, v') , so every triple mapping to this column has the form (u, e, v) with $e = (u', v')$. If the graph H contains the edge (u', v') , then in both outer multipliers this column contains the single edge variable $X_{(u', v')}$. Otherwise the column contains no edge variables.

Finally, we check the miter variables. Each miter variable e_ℓ depends only on the partial products of the two outer multipliers in the strip containing column ℓ . In each of the four cases, these partial products involve edge variables from at most one star, so each miter variable e_ℓ is also constant, edge-local, or star-local. $\square$

The next lemma is not used directly in the reduction, but it is needed in the proof of Lemma 8.6.

Lemma 8.5 (Inner multipliers impose perfect matching). *Under the restriction ρ , the inner multipliers impose exactly the perfect-matching constraints on the variables X_e : for every assignment α to the edge variables X_e , the propagated assignment $\bar{\alpha}$ (Definition 8.1) satisfies $\mathbf{xy} \upharpoonright_\rho$ if and only if α sets exactly one edge variable in the star $E(v)$ to true for every $v \in V$. Symmetrically, $\bar{\alpha}$ satisfies $\mathbf{yz} \upharpoonright_\rho$ if and only if α sets exactly one edge variable in the star $E(u)$ to true for every $u \in U$.*

Proof. Consider the multiplier $\mathbf{xy}$; the argument for $\mathbf{yz}$ is symmetric.

Suppose first that the assignment α selects exactly one edge in every star $E(v)$, $v \in V$. By Lemma 7.5(i), for each $v \in V$ the g -column $\text{col}_{\mathbf{xy}}(v)$ contains exactly the partial products $\{X_e : e \in E(v)\}$. Since ρ is g -sparse (Lemma 7.5(ii)), Observation 4.5 applies, and the strip based at this column outputs 1 in binary, agreeing with ρ . In a strip holding a single partial product X_e , the output bit at the base column computes X_e and is left unset by ρ , while the $g - 1$ higher output bits of the strip compute 0. Strips whose partial products are all forced to 0 output 0. In all cases the propagated values agree with ρ , so $\bar{\alpha}$ satisfies every clause of $\mathbf{xy} \upharpoonright_\rho$.

Conversely, suppose that the propagated assignment $\bar{\alpha}$ satisfies $\mathbf{xy} \upharpoonright_\rho$. Since $\bar{\alpha}$ extends ρ , it satisfies the unrestricted encoding $\mathbf{xy}$, and is therefore a correct evaluation of the multiplier (Definition 2.3). By Lemma 7.5(i), for each $v \in V$ the strip at base $\text{col}_{\mathbf{xy}}(v)$ outputs the binary count

of the true edge variables in $E(v)$. Since the restriction ρ sets this strip's output to 1 in binary, exactly one edge variable of $E(v)$ is true in the edge assignment α . $\square$

The reduction will use the following lemma to derive the clauses of $\text{Assoc}_n \upharpoonright_\rho$ from $\text{SPM}(H)$.

Lemma 8.6 (Local soundness). *Let C be a clause of $\text{Assoc}_n \upharpoonright_\rho$. If an assignment α to the edge variables satisfies the clauses $\text{ExactOne}(X_{E(w)})$ for every $w \in \text{supp}(C)$, then the propagated assignment $\bar{\alpha}$ (Definition 8.1) satisfies C .*

Proof. The clause C originates from one of three components: the inner multipliers $\mathbf{xy}$ and $\mathbf{yz}$, the outer multipliers $(\mathbf{xy})\mathbf{z}$ and $\mathbf{x}(\mathbf{yz})$, or the clauses defining the miter variables e_i . The restricted formula $\text{Assoc}_n \upharpoonright_\rho$ does not contain the long miter clause $e_0 \vee \dots \vee e_{4n-1}$ since ρ satisfies it by assigning $e_K = 1$. We check the cases in turn.

Inner multipliers. Suppose that C originates from the inner multiplier $\mathbf{xy}$; the case of $\mathbf{yz}$ is symmetric. Let α' be an assignment to the edge variables that agrees with α on the stars $E(v)$ with $v \in V \cap \text{supp}(C)$, and that selects exactly one edge in every remaining star $E(v)$, where $v \in V \setminus \text{supp}(C)$. This is well defined because the stars $E(v)$ partition the edges E . Since α satisfies $\text{ExactOne}(X_{E(v)})$ for every $v \in V \cap \text{supp}(C)$, the assignment α' in fact selects exactly one edge in every star $E(v)$, $v \in V$. By Lemma 8.5, the propagated assignment $\bar{\alpha}'$ satisfies every clause of $\mathbf{xy} \upharpoonright_\rho$, and in particular satisfies C .

Since every variable of C depends only on edge variables X_e with an endpoint $v_e \in \text{supp}(C)$, on which α' is defined to agree with α , the propagated assignments $\bar{\alpha}$ and $\bar{\alpha}'$ agree on the variables of C . Hence $\bar{\alpha}$ satisfies C as well.

Outer multipliers. The restriction ρ assigns no wires in the outer multipliers; it fixes only some of their inputs, namely bits of $\mathbf{x}$, $\mathbf{z}$, $\mathbf{xy}$, and $\mathbf{yz}$. The propagated assignment $\bar{\alpha}$ therefore satisfies every clause of the outer multipliers.

Miter clauses. The miter variables e_ℓ with $\ell \neq K$ are unassigned by ρ , and their defining clauses are gate definitions, so the propagated assignment $\bar{\alpha}$ satisfies them as well.

It remains to check the clauses defining e_K . As ρ sets $e_K = 1$, these clauses assert that the two outer multipliers disagree at output bit K . By Lemma 7.6, these two output bits are forced to disagree under ρ , so $\bar{\alpha}$ satisfies these clauses. $\square$

Lemma 8.7 (Reduction from perfect matching). *From any resolution refutation of Assoc_n one can obtain a refutation of the star-local extension $\text{SPM}(H)$ (Definition 5.2) with only a constant-factor increase in size.*

Proof. By the substitution lemma (Lemma 2.2), a size s refutation of Assoc_n yields a refutation of $\text{Assoc}_n \upharpoonright_\rho$ of size at most s .

Next, define a substitution σ from the variables of $\text{Assoc}_n \upharpoonright_\rho$ to constants and literals over the variables of $\text{SPM}(H)$ as follows. Every wire that is forced to a constant under ρ maps to that constant. The unset input bits $y_{j(e)}$ map to the edge variables X_e . Each edge-local wire maps to the literal it computes, X_e or $\neg X_e$. Every remaining wire is, by Lemma 8.4, a Boolean function f of a single star $E(w)$, and maps to the extension variable $Z_{w,f}$. Wires computing the same function of the same star all map to the same variable $Z_{w,f}$. By the substitution lemma (Lemma 2.2), the refutation of $\text{Assoc}_n \upharpoonright_\rho$ yields a refutation of $(\text{Assoc}_n \upharpoonright_\rho)^\sigma$ of size at most s .

It remains to derive the clauses of the restricted, substituted formula $(\text{Assoc}_n \upharpoonright_\rho)^\sigma$ from $\text{SPM}(H)$. Each clause C^σ of $(\text{Assoc}_n \upharpoonright_\rho)^\sigma$ is the result of applying the substitution σ to a clause C of the restricted formula $\text{Assoc}_n \upharpoonright_\rho$. The clause C has constant width: all clauses of Assoc_n have constant width except the long miter clause $e_0 \vee \dots \vee e_{4n-1}$, which ρ satisfies. By Lemma 8.4, each variable of C depends on the edge variables of at most one star, so $\text{supp}(C)$ has constant size.

We claim that the clauses $\text{ExactOne}(X_{E(w)})$ and $\text{Def}(Z_{w,f})$ for $w \in \text{supp}(C)$ imply C^σ . Indeed, let α be an assignment to the variables of $\text{SPM}(H)$ that satisfies these clauses. By Lemma 8.6, the propagated assignment $\bar{\alpha}$ (Definition 8.1) satisfies C . Furthermore, the definitions $\text{Def}(Z_{w,f})$ ensure that the substituted values agree with the propagated values, i.e., $\alpha(\sigma(q)) = \bar{\alpha}(q)$ for every variable q of C . Hence, α satisfies C^σ . By Lemma 5.7, some subclause of C^σ therefore has a constant-size derivation from $\text{SPM}(H)$. Applying the refutation of $(\text{Assoc}_n \upharpoonright_\rho)^\sigma$ to these subclauses gives a refutation of $\text{SPM}(H)$ of size $O(s)$. $\square$

Theorem 8.8 (Lower bound for multiplier associativity). *Let Mul_n be a strip-local multiplier encoding. For all sufficiently large n ,*

$$\text{size}_{\text{Res}}(\text{Assoc}_n) \geq 2^{\Omega((n/\log n)^{1/4})}.$$

Proof. Choose

$$m = \left\lfloor c_0(n/\log n)^{1/4} \right\rfloor$$

for a sufficiently small absolute constant c_0 . Theorem 5.4 provides a bipartite expander $H = (U, V, E)$ with $|U| = m + 1$ and $|V| = m$ whose maximum degree is at most a constant Δ , and no isolated vertices, as Section 5 requires. Theorem 5.6 then shows that every resolution refutation of $\text{SPM}(H)$ has size $2^{\Omega(m)}$. Lemma 7.3 gives $n_0 = O(m^4 \log m)$, so with a sufficiently small c_0 its construction fits into the n input bits; unused high input bits are set to zero.

By Lemma 8.7, a refutation of Assoc_n of size s gives a refutation of $\text{SPM}(H)$ of size $O(s)$. Hence $s \geq 2^{\Omega(m)} = 2^{\Omega((n/\log n)^{1/4})}$. $\square$

Remark 8.9 (Truncated multiplication). The above lower bound, as well as the bounded-depth Frege lower bound we will prove in Section 9, applies unchanged to associativity of truncated multiplication, where each multiplier outputs only the n low-order bits of the product and the miter compares n bits. The construction places the input positions $i(u), j(e), k(v)$, the restricted strip columns, and the miter variable e_K all below $n_0 \leq n$. So under ρ , every partial product not forced to 0 lies in a column below n_0 .

9 Bounded-depth Frege lower bound for Assoc_n

In this section we prove that Assoc_n is also hard for bounded-depth Frege systems. We reuse the reduction of Section 8 with the odd grid in place of the expander H . The source of hardness is now Håstad's lower bound for bounded-depth Frege refutations of the corresponding perfect-matching principle [Hå25]. Since Frege lines are formulas rather than clauses, we can directly substitute a constant-size formula for each wire, so the extension variables $Z_{w,f}$ from Section 5 are no longer needed.

9.1 Bounded-depth Frege systems

Formulas and depth. We follow the conventions of Håstad [Hå25]. Formulas are built from variables using negation and disjunctions of unbounded arity. Conjunctions are expressed as $\neg(\neg F_1 \vee \dots \vee \neg F_k)$. The *depth* of a formula is the maximum number of alternations between $\vee$ and $\neg$ on a path from its root to a leaf. Thus a literal has depth 0 and a clause has depth at most 1. The *size* of a formula is its number of symbols.

Frege systems. A *Frege system* is specified by a finite set of inference rules. Each rule has the form

$$\frac{F_1(p_1, \dots, p_k) \quad \dots \quad F_j(p_1, \dots, p_k)}{F(p_1, \dots, p_k)}$$

for fixed formulas $F_1, \dots, F_j$ and F over the variables $p_1, \dots, p_k$, and may be applied with arbitrary formulas substituted for $p_1, \dots, p_k$. A *derivation* of a formula G from a set of formulas Γ is a sequence of formulas ending in G , each of which belongs to Γ or follows from earlier formulas by an inference rule. The rules are required to be sound and *implicationally complete*, meaning that Γ implies G if and only if G has a derivation from Γ [CR79]. A *refutation* of a set of formulas Γ is a derivation of $\perp$ from Γ . The *depth* of a derivation is the maximum depth of its formulas, and its *size* is the total size of its formulas. We write $\text{size}_{\text{Frege}_d}(\Phi)$ for the minimum size of a Frege refutation of a set of formulas Φ of depth at most d .

We fix an arbitrary Frege system throughout this section. Håstad proved his lower bound for one specific system [Hå25, Section 2.2], but noted that the argument uses only that the rules are sound and of constant size, so his lower bound holds for every Frege system.

Formula substitutions. A *formula substitution* σ maps each variable to a formula. We write F^σ or π^σ for the result of applying σ to a formula F , or a derivation π , respectively. The *depth* and *size* of a substitution σ are the maximum depth and size of the formulas $\sigma(x)$. We write $\text{depth}(\cdot)$ and $|\cdot|$ for the depth and size of a formula, a derivation, or a substitution. Formula substitutions satisfy the following analogue of the substitution lemma for resolution (Lemma 2.2).

Lemma 9.1 (Frege substitution lemma, cf. [CR79, Lemma 2.5]). *If π is a derivation of G from Γ , then π^σ is a derivation of G^σ from Γ^σ of depth at most $\text{depth}(\pi) + \text{depth}(\sigma) + 1$ and size at most $|\sigma| \cdot |\pi|$.*

Proof. Frege rules are closed under substitution, so each inference of π is still an inference of π^σ . The depth grows by at most $\text{depth}(\sigma)$ plus one for a possible alternation where $\sigma(x)$ is substituted. $\square$

The following lemma is the Frege system analogue of Lemma 5.7, and follows immediately from implicational completeness.

Lemma 9.2 (Local consequences have constant-size Frege derivations). *If a set Γ of clauses over k variables implies a formula G , then G has a derivation from Γ whose depth and size depend only on k and $|G|$.*

9.2 The odd grid

For an odd integer $t \geq 3$, the *grid* Grid_t is the graph with vertex set $[1, t]^2$ in which two vertices are adjacent if they agree in one coordinate and differ by 1 in the other. Color the vertices as a chessboard, with a vertex (i, j) white if $i + j$ is even and black otherwise. Every edge joins a white vertex to a black vertex, so Grid_t is bipartite with sides

$$U = \{\text{white vertices}\}, \quad V = \{\text{black vertices}\}, \quad |U| = \frac{t^2+1}{2}, \quad |V| = \frac{t^2-1}{2},$$

and every vertex has degree at most 4. Thus Grid_t satisfies the standing assumptions of Section 5: it is a bipartite graph $H = (U, V, E)$ with $|U| = m + 1$ and $|V| = m$, where $m = (t^2 - 1)/2$, and has maximum degree at most $\Delta = 4$. We may therefore apply the reduction of Section 8 with $H = \text{Grid}_t$.

Håstad [Hå25] studied the CNF $\text{PM}(\text{Grid}_t)$ under the name of the *functional and onto pigeonhole principle* on the odd grid. From this perspective the white vertices are the pigeons and the black vertices are the holes.

Theorem 9.3 (Håstad [Hå25, Theorem 5.1]). *Let t be an odd integer and let $d \leq O(\log t / \log \log t)$. Then any depth d Frege refutation of $\text{PM}(\text{Grid}_t)$ requires size*

$$\exp(\Omega(t^{1/(2d-1)}(\log t)^{O(1)})).$$

In particular, there is a constant $c > 0$ such that for every odd t and every $d \leq c \log t / \log \log t$,

$$\text{size}_{\text{Frege}_d}(\text{PM}(\text{Grid}_t)) \geq \exp(t^{c/d}).$$

9.3 Reduction and lower bound

We now carry out the reduction of Section 8 in a bounded-depth Frege system. As in Section 8, we assume that $n \geq n_0$, and Assoc_n uses a strip-local multiplier encoding Mul_n .

Lemma 9.4 (Reduction from $\text{PM}(H)$). *There are constants d_0 and s_0 such that for every d , if π is a depth d Frege refutation of Assoc_n , then $\text{PM}(H)$ has a depth $d + d_0$ Frege refutation of size at most $s_0 |\pi|$.*

Proof. This reduction follows the same three steps as the resolution reduction (Lemma 8.7). The main difference is that here, in order to use Lemma 9.1, we apply ρ as a formula substitution, which does not simplify away constants as a restriction would (otherwise an inference may become invalid). In order to reuse the lemmas from Section 8, we will therefore need to derive the formulas of Assoc_n^ρ (where ρ is applied as a formula substitution) from the clauses of $\text{Assoc}_n \upharpoonright_\rho$ (where ρ is applied as a restriction). The substitution and local-derivation steps are essentially unchanged, except that formulas take the place of extension variables.

Application of ρ . Let π be a depth d Frege refutation of Assoc_n of size s . By Lemma 9.1, applying ρ as a formula substitution gives a depth $d + O(1)$, size $O(s)$ refutation π^ρ of Assoc_n^ρ . Aside from the long miter clause $e_0 \vee \dots \vee e_{4n-1}$, each clause C^ρ of Assoc_n^ρ has constant size, and hence has a derivation of constant depth and size (Lemma 9.2) from its counterpart $C \upharpoonright_\rho$ in $\text{Assoc}_n \upharpoonright_\rho$ (if the clause C is satisfied by ρ then we instead derive C^ρ by weakening from $\neg \perp$). The long miter clause after applying the formula substitution ρ becomes $e_0 \vee \dots \vee e_{K-1} \vee \neg \perp \vee e_{K+1} \vee \dots \vee e_{4n-1}$. We derive this in constant depth and size $O(n)$ as follows: fix a constant-size derivation of $p \vee q$ from p , then substitute $\neg \perp$ for p and $e_0 \vee \dots \vee e_{K-1} \vee e_{K+1} \vee \dots \vee e_{4n-1}$ for q .

Prepending these derivations to π^ρ gives a refutation $\pi \upharpoonright_\rho$ of $\text{Assoc}_n \upharpoonright_\rho$ of depth $d + O(1)$ and size $O(s)$.

DNF substitution. By Lemma 8.4, every variable of $\text{Assoc}_n \upharpoonright_\rho$ is an edge variable or a wire that is constant, edge-local, or star-local. Define a formula substitution σ accordingly: an edge variable X_e maps to itself; a constant wire maps to the corresponding value $\neg \perp$ or $\perp$; an edge-local wire maps to the corresponding literal X_e or $\neg X_e$; and a wire that is star-local at w and computes $f(X_{E(w)})$ maps to the canonical DNF of f , which has depth at most 3 and constant size. Then σ is a substitution with depth at most 3 and constant size. By Lemma 9.1, $(\pi \upharpoonright_\rho)^\sigma$ is a depth $d + O(1)$, size $O(s)$ refutation of $(\text{Assoc}_n \upharpoonright_\rho)^\sigma$.

Local derivations. It remains to derive the formulas of $(\text{Assoc}_n \upharpoonright_\rho)^\sigma$ from $\text{PM}(H)$. Each such formula is C^σ for a clause C of $\text{Assoc}_n \upharpoonright_\rho$. The clause C has constant width, since ρ satisfies the long miter clause, and by Lemma 8.4 each variable of C depends on the edge variables of at most one star, so $\text{supp}(C)$ has constant size. We claim that the clauses $\text{ExactOne}(X_{E(w)})$ for $w \in \text{supp}(C)$

imply C^σ . Indeed, let α be an assignment to the edge variables that satisfies these clauses. By Lemma 8.6, the propagated assignment $\bar{\alpha}$ (Definition 8.1) satisfies the clause C . Furthermore, σ maps each variable q of clause C to a formula computing its Boolean function, so the substituted values agree with the propagated values, i.e., $\alpha(\sigma(q)) = \bar{\alpha}(q)$. Hence α satisfies C^σ . By Lemma 9.2, the formula C^σ therefore has a constant-depth, constant-size derivation from $\text{PM}(H)$. Applying the refutation $(\pi \upharpoonright_\rho)^\sigma$ of $(\text{Assoc}_n \upharpoonright_\rho)^\sigma$ to these derived formulas gives a refutation of $\text{PM}(H)$ with depth $d + O(1)$ and size $O(s)$. $\square$

Theorem 9.5 (Bounded-depth Frege lower bound). *Let Mul_n be a strip-local multiplier encoding. There is a constant $\gamma > 0$, depending only on the encoding, such that for all sufficiently large n and all $d \leq \gamma \log n / \log \log n$,*

$$\text{size}_{\text{Frege}_d}(\text{Assoc}_n) \geq \exp(n^{\gamma/d}).$$

Proof. Lemma 7.3 gives a value $n_0 = O(m^4 \log m) = O(t^8 \log t)$. Let $n_0(t)$ denote this value as a function of t . Set t to the largest odd integer such that $n_0(t) \leq n$. Then $n_0(t+2) > n$, therefore $t = \Omega((n/\log n)^{1/8})$. So when n is sufficiently large, we have $t \geq n^{1/9}$.

Let c be the constant of Håstad's lower bound (Theorem 9.3), and let d_0 and s_0 be the constants of Lemma 9.4. Set the constant $\gamma = c/(18(1+d_0))$.

Now let $d \leq \gamma \log n / \log \log n$, and suppose that Assoc_n has a depth d Frege refutation of size s . Applying the reduction of Lemma 9.4 yields a Frege refutation of $\text{PM}(\text{Grid}_t)$ of depth $d + d_0$ and size at most $s_0 s$. Since $\gamma \leq c/18$ and $t \geq n^{1/9}$, we have $d + d_0 \leq c \log t / \log \log t$ for all sufficiently large n . Therefore we may apply Theorem 9.3 with depth $d + d_0 \leq (1 + d_0)d$, which gives

$$s_0 s \geq \exp(t^{c/((1+d_0)d)}) \geq \exp(n^{c/(9(1+d_0)d)}) = \exp(n^{2\gamma/d}).$$

Finally, since $d \leq \gamma \log n / \log \log n$, the exponent $n^{2\gamma/d}$ exceeds $n^{\gamma/d}$ by at least a factor of $\log n$, which absorbs s_0 for all sufficiently large n . Hence $s \geq \exp(n^{\gamma/d})$. $\square$

Remark 9.6. A resolution refutation is a depth 1 Frege refutation in a Frege system whose rules include resolution, so Theorem 9.5 with $d = 1$ yields a resolution lower bound of $\exp(n^\gamma)$. However, the proof gives $\gamma < 1/18$, so this bound is weaker than the lower bound of $2^{\Omega((n/\log n)^{1/4})}$ given by Theorem 8.8.

Remark 9.7 (Number of lines). Håstad also proved that for $d \leq O(\log t / \log \log t)$, if every line of a depth d Frege refutation of $\text{PM}(\text{Grid}_t)$ has size at most M , then the refutation has at least $\exp(\Omega(t/((\log M)^{2d-1}(\log t)^{O(1)})))$ lines [Hå25, Theorem 7.1]. The proof of Lemma 9.4 turns a depth d Frege refutation of Assoc_n with L lines, each of size at most M , into a depth $d + d_0$ Frege refutation of $\text{PM}(\text{Grid}_t)$ with $O(L)$ lines of size $O(M)$, so L satisfies the same bound with $d + d_0$ in place of d , $O(M)$ in place of M , and $t = \Omega((n/\log n)^{1/8})$. In particular, for constant d and lines of polynomial size, a depth d Frege refutation of Assoc_n requires $\exp(n^{1/8-o(1)})$ lines.

10 Conclusion and open problems

In this paper we resolve an open problem posed by Beame and Liew in [BL19] by proving an exponential lower bound on the size of resolution refutations of the multiplier associativity formula Assoc_n (Theorem 8.8). Using the same method, we also obtain an exponential lower bound for bounded-depth Frege systems (Theorem 9.5). These lower bounds apply to any strip-local multiplier encoding, which includes array and Wallace-tree multipliers. The resolution lower bound closes off a possibility raised in the introduction: for associativity, at least, there are no polynomial size resolution proofs that CDCL solvers merely fail to find.

Beyond the lower bounds themselves, our reduction uncovers a connection between two seemingly unrelated principles: the associativity of grade-school multiplication and perfect matching in bipartite graphs.

An immediate open problem is to improve the resolution lower bound quantitatively. The exponent $(n/\log n)^{1/4}$ in Theorem 8.8 reflects the cost of our embedding (Lemma 7.3), which requires $O(m^4 \log m)$ input bits to embed an m -vertex matching instance; a denser embedding could sharpen the exponent. Moreover, the constants in our bound do not give concrete hardness at practical bit-widths such as $n = 64$. In this respect the pigeonhole principle is similar, since the proven bounds [Hak85, BP96] are vacuous at the sizes where solvers actually fail. A lower bound of this kind nevertheless addresses the empirical hardness at the right level: it shows that exponential scaling is unavoidable for resolution, and hence for CDCL solvers, no matter how good their heuristics. We leave open whether Assoc_n requires resolution refutations of size $2^{\Omega(n)}$, and whether the hardness can be made concrete at practical bit-widths.

It also remains open to determine the proof complexity of multiplier associativity in polynomial calculus (with or without resolution) and in cutting planes. Our reduction yields no lower bound in either case, since the perfect-matching principle has small proofs in both. Nevertheless, we conjecture that (bit-level) multiplier associativity remains hard for these systems as well.

Another open line of work is to prove lower bounds for the associativity of other multiplier architectures. Our lower bounds do not apply to Booth multipliers, which are based on partial product summation but are not strip-local, nor to Karatsuba, Toom–Cook, and Schönhage–Strassen multipliers, which are not based on partial product summation.

Finally, we conjecture that the $O(n^6 \log n)$ -size resolution proofs of commutativity and distributivity found by Beame and Liew [BL19] are asymptotically optimal. A matching lower bound would show that SAT solvers inherently struggle with these identities, since proofs of this size are already prohibitive at practical bit-widths.

Acknowledgements

We thank Paul Beame for suggesting a simplified proof of Theorem 5.6 and for other helpful comments and pointers to the literature, and thank Andrei Constantinescu for suggestions that improved the exposition of Section 5.

A Wallace-tree multipliers are strip-local

We first define the carry-lookahead adder.

Definition A.1 (Carry-lookahead adder). A *carry-lookahead adder* (CLA) uses a tree structure to add two bit-vectors $\mathbf{x}, \mathbf{y}$ with only logarithmic depth. The 4-bit CLA computes, for each pair x_i, y_i , the *generate* and *propagate* signals

$$q_i = x_i y_i, \quad p_i = x_i \oplus y_i.$$

Then, writing c_i for the carry bit in the i -th column, we have

$$c_{i+1} = q_i \oplus (p_i c_i),$$

with carry-in $c_0 = 0$. Unrolling this recurrence gives the following equations, which compute each carry bit in parallel from the values q_i, p_i and c_0 :

$$c_1 = q_0 \oplus p_0 c_0$$

$$\begin{aligned}
c_2 &= q_1 \oplus q_0 p_1 \oplus c_0 p_0 p_1 \\
c_3 &= q_2 \oplus q_1 p_2 \oplus q_0 p_1 p_2 \oplus c_0 p_0 p_1 p_2 \\
c_4 &= q_3 \oplus q_2 p_3 \oplus q_1 p_2 p_3 \oplus q_0 p_1 p_2 p_3 \oplus c_0 p_0 p_1 p_2 p_3.
\end{aligned}$$

These values are used to compute the outputs $(x + y)_i = c_i \oplus x_i \oplus y_i$. The 4-bit CLA additionally computes the *group propagate* and *group generate*

$$p^1 = p_0 p_1 p_2 p_3, \quad q^1 = q_3 \oplus q_2 p_3 \oplus q_1 p_2 p_3 \oplus q_0 p_1 p_2 p_3,$$

where the superscript indicates the level.

We construct a 16-bit CLA with two levels, whose first half is shown in Figure 9. At the zeroth level we arrange four 4-bit CLAs, the k -th CLA taking inputs $x_i, y_i, i \in [4k, 4k + 4)$. We denote the k -th CLA's group propagate and generate by p_{4k}^1, q_{4k}^1 . Then the carries c_4, c_8, c_{12}, c_{16} can be computed by the equations

$$\begin{aligned}
c_4 &= q_0^1 \oplus p_0^1 c_0 \\
c_8 &= q_4^1 \oplus q_0^1 p_4^1 \oplus c_0 p_0^1 p_4^1 \\
c_{12} &= q_8^1 \oplus q_4^1 p_8^1 \oplus q_0^1 p_4^1 p_8^1 \oplus c_0 p_0^1 p_4^1 p_8^1 \\
c_{16} &= q_{12}^1 \oplus q_8^1 p_{12}^1 \oplus q_4^1 p_8^1 p_{12}^1 \oplus q_0^1 p_4^1 p_8^1 p_{12}^1 \oplus c_0 p_0^1 p_4^1 p_8^1 p_{12}^1.
\end{aligned}$$

Notice that these equations are isomorphic to the previous equations for computing carries within each 4-bit CLA. We can reuse the same circuitry from the 4-bit CLA to compute these carries, as well as the group propagate and generate for the next level. We can repeat this process to construct larger CLAs, with each iteration able to handle four times the bit-width.

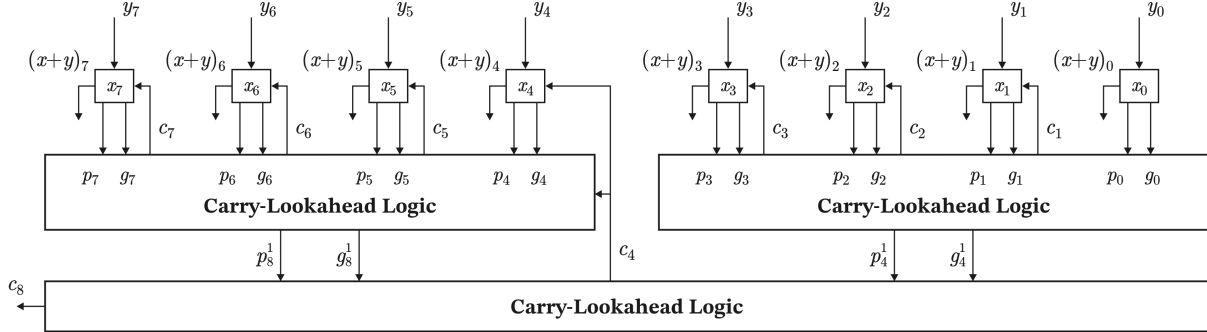


Figure 9: An 8-bit carry-lookahead adder built from two 4-bit CLAs. Each bit cell combines x_i and y_i into its signal pair (p_i, q_i) and sum bit. Each CLA computes the carries into its four cells and feeds its group pair, written (p^1, q^1) , to the second-level circuitry, which computes the carries c_4 and c_8 .

We next define the Wallace-tree multiplier. Figure 10 shows a 9-bit instance.

Definition A.2 (Wallace-tree multiplier). A *Wallace-tree multiplier* takes input $(\mathbf{x}, \mathbf{y})$ and computes each partial product of Definition 2.4 by an AND gate. It then goes through $\ell_{\max} < \log_{3/2} n$ layers of parallel adders to reduce the n starting rows of partial products to two rows of equivalent arithmetic weight.

We define layer variables $t_{\ell, i, j}$, where ℓ is the layer, i is the column containing the variable, and j is the row (note that this differs from the convention for a partial product $t_{i, j}$, whose subscripts

are input positions and whose column is $i + j$). A variable in column i has arithmetic weight 2^i . We denote the *subcolumn* of column i within layer ℓ by

$$\text{Col}(\ell, i) = \{t_{\ell, i, j} \text{ for all } j\}.$$

In the zeroth layer, the variables are the partial products, arranged so that $\text{Col}(0, i)$ holds the partial products of column i :

$$t_{0, i, j} = x_{i-j} \wedge y_j \quad \text{for } i < n, \quad t_{0, i, j} = x_{n-1-j} \wedge y_{i-n+j+1} \quad \text{for } i \geq n.$$

We now specify how to construct layer $\ell + 1$ from layer ℓ . We partition the rows of layer ℓ into sets of three, from top to bottom. Adder $A_{\ell, i, j}$ takes as input the variables in the i -th column of the j -th set of three rows, replacing nonexistent variables with the constant 0. On inputs a_0, a_1, a_2 it outputs the sum bit $s = a_0 \oplus a_1 \oplus a_2$ and the carry bit $c = \text{MAJ}(a_0, a_1, a_2)$. For each set of rows $j = 0, 1, \dots$ and each column i , we append adder $A_{\ell, i, j}$'s sum bit to subcolumn $\text{Col}(\ell + 1, i)$ and its carry bit to subcolumn $\text{Col}(\ell + 1, i + 1)$. An adder with no constant inputs is the full adder of Section 2; an adder with exactly one constant input is a *half adder*, computing $s = a_0 \oplus a_1$ and $c = a_0 \wedge a_1$; an adder with two constant inputs acts as a wire.

Each layer reduces the number of rows from N to $\lceil 2N/3 \rceil$, leaving the last layer $\ell_{\max}$ with only two rows. Every adder has the same total weight of inputs and outputs, so each layer has the same weighted sum $\sum_{i, j} t_{\ell, i, j} 2^i$ as the partial products. Using a $2n$ -bit carry-lookahead adder (Definition A.1), we sum these two rows, outputting the product in the multiplier's output bits.

Observation A.3. *The Wallace-tree multiplier is an exact multiplier encoding (Definition 2.3).*

Lemma A.4 (Wallace-tree strip-locality). *Fix $g \geq 1$ and a g -sparse restriction of the input bit-vectors. Then every layer variable of the Wallace tree is constant or a Boolean function of the partial products of the strip containing it, and every layer variable in the top column of a strip is the constant 0.*

Proof. In the Wallace tree, the only wires crossing a column boundary are adder carries, so like in the proof of Proposition 4.6 it suffices to show that every carry entering a strip is the constant 0. This is trivial for the strip at base 0.

Assume the inductive hypothesis that every carry entering the strip with base bg is zero. Then every layer variable $t_{\ell, i, j}$ in the strip is a function of the strip's partial products not forced to 0. By g -sparsity, the strip contains fewer than 2^{g-1} such partial products, and they are all contained in the base column bg , so their total weight is less than $2^{g-1} \cdot 2^{bg} = 2^{(b+1)g-1}$. Under any assignment to these partial products, the strip's weighted sum can only decrease from one layer to the next, from carries going out of the strip's top column. The strip's weighted sum therefore stays below $2^{(b+1)g-1}$ in every layer, and hence every variable in the strip's top column $(b+1)g - 1$ is 0. In particular every adder in the top column has all inputs 0 and so every carry entering the next strip is 0. $\square$

Lemma A.5 (Carry-lookahead adder strip-locality). *Fix $g \geq 1$ and let a carry-lookahead adder (Definition A.1) add two bit-vectors $\mathbf{x}, \mathbf{y}$ whose bits in the top column of every g -strip are the constant 0. Then every wire of the adder is constant or a Boolean function of the summand bits in the columns of a single strip.*

Proof. Let p_i, q_i and c_i denote the bitwise signals and carries of the carry-lookahead adder (Definition A.1). At the top column s of every strip we have $x_s = y_s = 0$, hence $p_s = q_s = 0$ and $c_{s+1} = 0$.

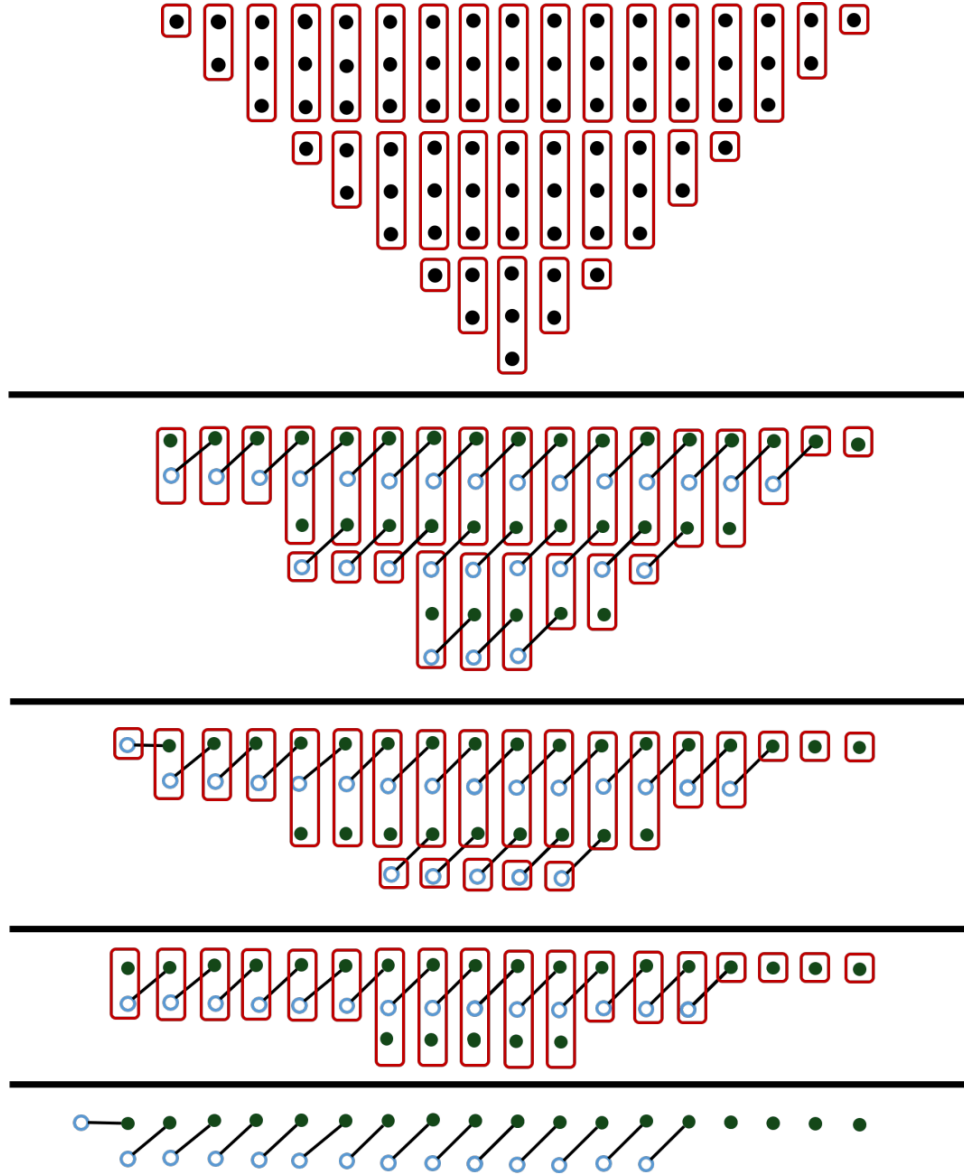


Figure 10: Dot diagram for a 9×9 Wallace-tree multiplier. Hollow dots represent carry bits and solid dots represent sum bits. Dots connected by an edge are output by the same adder.

It remains to show that the wires of the higher levels, which compute the carries through group pairs rather than through the carry recurrence, also depend only on the columns of a single strip. Unwinding the carry recurrence over a contiguous interval $I = [a, i]$ shows that its carry-out is $c_{i+1} = Q_I \oplus (P_I \wedge c_a)$, where

$$P_I = \bigwedge_{t=a}^i p_t, \quad Q_I = \bigoplus_{t=a}^i \left(q_t \wedge \bigwedge_{u=t+1}^i p_u \right).$$

Suppose I contains a top column, and let s be the highest top column it contains. Then $p_s = q_s = 0$, so $P_I = 0$, and every term of Q_I with $t \leq s$ vanishes, since it contains the factor q_s if $t = s$ and the factor p_s if $t < s$. Consequently (P_I, Q_I) depends only on the signal pairs (p_t, q_t) with $t > s$, which lie in the strip containing column i .

We now prove the lemma for each type of wire in the carry-lookahead adder: the bitwise signals, group pairs, carries, and sum bits. A bitwise signal p_i or q_i depends only on the summand bits of column i . A group pair (p_a^ℓ, q_a^ℓ) equals (P_I, Q_I) for the interval $I = [a, a + 4^\ell)$ spanned by its level- ℓ sub-adder, and is therefore constant or depends only on the signal pairs of a single strip. A carry wire computes some carry c_i of the recurrence, and taking $I = [0, i - 1]$ with $c_0 = 0$ gives $c_i = Q_{[0, i-1]}$. Since $c_i = 0$ whenever $i - 1$ is a top column, we may assume that $i - 1$ and i lie in the same strip. Then, from the previous paragraph, c_i is constant or depends only on the signal pairs of this strip. This implies that a sum bit $(x + y)_i = p_i \oplus c_i$ also depends only on the summand bits of the strip containing i . $\square$

Proposition 4.7. *The standard Wallace-tree multiplier encoding (Definition A.2), where the final two rows are summed by a carry-lookahead adder (Definition A.1), is strip-local.*

Proof. Every gate of the multiplier has constant fan-in, so the clauses of the encoding have constant width. To verify locality, fix $g \geq 1$ and a g -sparse restriction. By Lemma A.4, the bits of the two final rows in every strip's top column are the constant 0, so Lemma A.5 applies to the final carry-lookahead adder. Together, the two lemmas show that every wire of the multiplier is constant or a Boolean function of the partial products belonging to a single strip. $\square$

B Lean formalization

We provide a Lean 4 formalization of the main theorems, generated by Claude Fable 5.1, at <https://github.com/dysfunctor/associativity>. We are not aware of an established library for proof complexity or arithmetic circuits, so this formalization includes an implementation of these definitions. While we have inspected these definitions ourselves, we caution that they are a potential source of error.

AI Methodology

The main ideas and the core proof of the associativity lower bound are due to GPT-5.6-Sol. This materially affected Sections 5, 7, 8, and 9, and Appendix A. During the verification process, the author reformulated and rewrote the definitions, lemma statements, and proofs with the assistance of Claude Fable 5, and added attributions along the way. The author wrote initial drafts of the expository text, including the introduction, proof outline, and conclusion, and used Claude Fable 5 afterwards to improve presentation.

The author produced the diagrams with assistance from Claude Fable 5. The author drew each figure by hand, sent a photo to Claude, and then refined the generated diagram with further prompting.

The Lean formalization (Appendix B) was generated by Claude Fable 5.1.

The author verified the correctness and originality of all content, including references, and takes full responsibility for the paper’s content.

References

- [AB04] Albert Atserias and María Luisa Bonet, *On the Automatizability of Resolution and Related Propositional Proof Systems*, Information and Computation **189** (2004), no. 2, 182–201.
- [ABRW04] Michael Alekhnovich, Eli Ben-Sasson, Alexander A. Razborov, and Avi Wigderson, *Pseudorandom Generators in Propositional Proof Complexity*, SIAM Journal on Computing **34** (2004), no. 1, 67–88.
- [BBB⁺22] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar, *cvc5: A Versatile and Industrial-Strength SMT Solver*, Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2022), Part I, Lecture Notes in Computer Science, vol. 13243, 2022, pp. 415–442.
- [BBK⁺26] Armin Biere, Martin Brain, Daniel Kroening, Norbert Manthey, and Michael Tautschnig, *Proof-Guided SAT Encoding Selection for Bit-Vector Multiplication*, Proceedings of the Workshop on Pragmatics of SAT (POS 2026), co-located with FLoC 2026, Lisbon, 2026.
- [BFFH20] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximilian Heisinger, *CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling Entering the SAT Competition 2020*, Proceedings of SAT Competition 2020 – Solver and Benchmark Descriptions, Department of Computer Science Series of Publications B, vol. B-2020-1, University of Helsinki, 2020, pp. 50–53.
- [Bie16] Armin Biere, *Weaknesses of CDCL Solvers*, Talk at the Fields Institute Workshop on Theoretical Foundations of SAT Solving, Toronto, August 2016, <http://www.fields.utoronto.ca/talks/weaknesses-cdcl-solvers>.
- [BKKS25] Armin Biere, Alexander Konrad, Daniela Kaufmann, and Christoph Scholl, *Split-BDD Multiplier Verification Benchmarks Submitted to the SAT Competition 2025*, Proceedings of SAT Competition 2025 – Solver and Benchmark Descriptions, Proceedings of SAT Competitions, TU Wien Academic Press, Vienna, 2025.
- [BKS04] Paul Beame, Henry Kautz, and Ashish Sabharwal, *Towards Understanding and Harnessing the Potential of Clause Learning*, Journal of Artificial Intelligence Research **22** (2004), 319–351.
- [BL19] Paul Beame and Vincent Liew, *Toward Verifying Nonlinear Integer Arithmetic*, Journal of the ACM **66** (2019), no. 3, Article 22, 1–30.

- [BN11] Eli Ben-Sasson and Jakob Nordström, *Understanding Space in Proof Complexity: Separations and Trade-offs via Substitutions*, Innovations in Computer Science (ICS 2011), 2011, pp. 401–416. Full version: arXiv:1008.1789.
- [BP96] Paul Beame and Toniann Pitassi, *Simplified and Improved Resolution Lower Bounds*, Proceedings of the 37th Annual Symposium on Foundations of Computer Science (FOCS 1996), 1996, pp. 274–282.
- [Bra93] Daniel Brand, *Verification of Large Synthesized Designs*, Proceedings of the 1993 International Conference on Computer Aided Design (ICCAD 1993), 1993, pp. 534–537.
- [BW01] Eli Ben-Sasson and Avi Wigderson, *Short Proofs Are Narrow—Resolution Made Simple*, Journal of the ACM **48** (2001), no. 2, 149–169.
- [CR79] Stephen A. Cook and Robert A. Reckhow, *The Relative Efficiency of Propositional Proof Systems*, Journal of Symbolic Logic **44** (1979), no. 1, 36–50.
- [dMB08] Leonardo de Moura and Nikolaj Bjørner, *Z3: An Efficient SMT Solver*, Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008), Lecture Notes in Computer Science, vol. 4963, 2008, pp. 337–340.
- [ES04] Niklas Eén and Niklas Sörensson, *An Extensible SAT-solver*, Theory and Applications of Satisfiability Testing (SAT 2003), Lecture Notes in Computer Science, vol. 2919, 2004, pp. 502–518.
- [ET41] Paul Erdős and Paul Turán, *On a Problem of Sidon in Additive Number Theory, and on Some Related Problems*, Journal of the London Mathematical Society **s1-16** (1941), no. 4, 212–215.
- [FLS⁺19] Yu-Fu Fu, Jiaxiang Liu, Xiaomu Shi, Ming-Hsien Tsai, Bow-Yaw Wang, and Bo-Yin Yang, *Signed Cryptographic Program Verification with Typed CryptoLine*, Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS 2019), 2019, pp. 1591–1606.
- [Hak85] Armin Haken, *The Intractability of Resolution*, Theoretical Computer Science **39** (1985), 297–308.
- [Hå25] Johan Håstad, *On Small-Depth Frege Proofs for PHP*, TheoretiCS **4** (2025), Article 27, 1–42.
- [IOSS16] Dmitry Itsykson, Vsevolod Oparin, Mikhail Slabodkin, and Dmitry Sokolov, *Tight Lower Bounds on the Resolution Complexity of Perfect Matching Principles*, Fundamenta Informaticae **145** (2016), no. 3, 229–242.
- [ISS15] Dmitry Itsykson, Mikhail Slabodkin, and Dmitry Sokolov, *Resolution Complexity of Perfect Matching Principles for Sparse Graphs*, Computer Science – Theory and Applications (CSR 2015), Lecture Notes in Computer Science, vol. 9139, 2015, pp. 219–230. Full version: ECCC Technical Report TR14-093, 2014.
- [KBBN22] Daniela Kaufmann, Paul Beame, Armin Biere, and Jakob Nordström, *Adding Dual Variables to Algebraic Reasoning for Gate-Level Multiplier Verification*, Proceedings of the 2022 Design, Automation & Test in Europe Conference (DATE 2022), 2022, pp. 1431–1436.

- [KBK19] Daniela Kaufmann, Armin Biere, and Manuel Kauers, *Verifying Large Multipliers by Combining SAT and Computer Algebra*, Proceedings of the 19th Conference on Formal Methods in Computer-Aided Design (FMCAD 2019), 2019, pp. 28–36.
- [KBK20] Daniela Kaufmann, Armin Biere, and Manuel Kauers, *SAT, Computer Algebra, Multipliers*, Vampire 2018 and Vampire 2019: The 5th and 6th Vampire Workshops, EPiC Series in Computing, vol. 71, 2020, pp. 1–18.
- [LBD⁺20] Vincent Liew, Paul Beame, Jo Devriendt, Jan Elffers, and Jakob Nordström, *Verifying Properties of Bit-vector Multiplication Using Cutting Planes Reasoning*, Proceedings of the 20th Conference on Formal Methods in Computer-Aided Design (FMCAD 2020), 2020, pp. 194–204.
- [Lie20] Vincent Liew, *A Path Paved by Proof Complexity Towards Verifying Nonlinear Integer Arithmetic*, Ph.D. dissertation, University of Washington, 2020.
- [MGD19] Alireza Mahzoon, Daniel Große, and Rolf Drechsler, *RevSCA: Using Reverse Engineering to Bring Light into Backward Rewriting for Big and Dirty Multipliers*, Proceedings of the 56th Annual Design Automation Conference (DAC 2019), 2019, Article 185.
- [NP23] Aina Niemetz and Mathias Preiner, *Bitwuzla*, Computer Aided Verification (CAV 2023), Part II, Lecture Notes in Computer Science, vol. 13965, 2023, pp. 3–17.
- [NPR⁺21] Aina Niemetz, Mathias Preiner, Andrew Reynolds, Yoni Zohar, Clark Barrett, and Cesare Tinelli, *Towards Satisfiability Modulo Parametric Bit-vectors*, Journal of Automated Reasoning **65** (2021), no. 7, 1001–1025.
- [NPZ24] Aina Niemetz, Mathias Preiner, and Yoni Zohar, *Scalable Bit-Blasting with Abstractions*, Computer Aided Verification (CAV 2024), Part I, Lecture Notes in Computer Science, vol. 14681, 2024, pp. 178–200.
- [OKTB23] Alex Ozdemir, Gereon Kremer, Cesare Tinelli, and Clark Barrett, *Satisfiability Modulo Finite Fields*, Computer Aided Verification (CAV 2023), Part II, Lecture Notes in Computer Science, vol. 13965, 2023, pp. 163–186.
- [RBK17] Daniela Ritirc, Armin Biere, and Manuel Kauers, *Column-Wise Verification of Multipliers Using Computer Algebra*, Proceedings of the 17th Conference on Formal Methods in Computer-Aided Design (FMCAD 2017), 2017, pp. 23–30.
- [REK⁺25] Jakob Rath, Clemens Eisenhofer, Daniela Kaufmann, Nikolaj Bjørner, and Laura Kovács, *PolySAT: Word-level Bit-vector Reasoning in Z3*, Verified Software: Theories, Tools, and Experiments (VSTTE 2024), Lecture Notes in Computer Science, vol. 15525, 2025, pp. 47–69.
- [SGK⁺16] Amr A. R. Sayed-Ahmed, Daniel Große, Ulrich Kühne, Mathias Soeken, and Rolf Drechsler, *Formal Verification of Integer Multipliers by Combining Gröbner Basis with Logic Reduction*, Proceedings of the 2016 Design, Automation & Test in Europe Conference (DATE 2016), 2016, pp. 1048–1053.
- [SLVG26] Sunidhi Singh, Vincent Liew, Marc Vinyals, and Vijay Ganesh, *Proofdoors and Efficiency of CDCL Solvers*, arXiv:2603.26286, 2026.

- [ZIM⁺22] Yoni Zohar, Ahmed Irfan, Makai Mann, Aina Niemetz, Andres Nötzli, Mathias Preiner, Andrew Reynolds, Clark Barrett, and Cesare Tinelli, *Bit-Precise Reasoning via Int-Blasting*, Verification, Model Checking, and Abstract Interpretation (VMCAI 2022), Lecture Notes in Computer Science, vol. 13182, 2022, pp. 496–518.